

RESEARCH ARTICLE

A Steady-State Eulerian Smoothed Particle Hydrodynamics (SPH) Approach for Incompressible Flow and Heat Transfer Using the Semi-Implicit Method for Pressure-Linked Equations (SIMPLE) Algorithm

Tae Hwan Kim¹  | Hee Sang Yoo^{1,2} | Jin-Woo Kim¹ | Eung Soo Kim¹

¹Department of Nuclear Engineering, Seoul National University, Seoul, South Korea | ²CEA, DES, IRESNE, Cadarache, Saint Paul-Lez-Durance, France

Correspondence: Eung Soo Kim (kes7741@snu.ac.kr)

Received: 23 June 2025 | **Revised:** 12 December 2025 | **Accepted:** 29 December 2025

Keywords: incompressible flow | SIMPLE algorithm | smoothed particle hydrodynamics (SPH) | steady state solver

ABSTRACT

A steady state Eulerian smoothed particle hydrodynamics (SPH) solver is proposed by integrating the Semi-Implicit Method for Pressure-Linked Equations (SIMPLE) algorithm. By removing time dependent terms from the governing equations, the proposed approach directly solves for steady-state velocity, pressure, and temperature fields for incompressible flows using a matrix-based formulation. To efficiently handle the resulting large, sparse linear systems, a matrix-free Bi-CGSTAB iterative solver is employed and the overall computational algorithm is fully parallelized on GPUs to accelerate performance. The accuracy and efficiency of the proposed steady solver are validated through several benchmark tests, including pipe flow with and without obstacles, 2D/3D lid-driven cavity flow with and without heat transfer, and natural convection flow. Compared to conventional transient Eulerian SPH solvers, the proposed method achieves 8.97–17.36 times speedup while maintaining high accuracy, making it a promising tool for steady state CFD analysis and as a precursor to transient simulations.

1 | Introduction

Smoothed particle hydrodynamics (SPH) is a mesh-free, inherently Lagrangian particle method originally developed in the field of astrophysics [1–3]. It has since gained widespread adoption in computational fluid dynamics (CFD), owing to its inherent ability to handle complex interface dynamics and large deformations, and has emerged as one of the most prominent meshless methods in modern numerical simulation [4–7]. Inherently as a Lagrangian approach, SPH offers significant advantages in tracking complex interface dynamics, large deformation phenomena, and free-surface flows [6–8]. These characteristics make SPH particularly well-suited for transient phenomena such as sloshing, wave breaking,

and multiphase flow [6, 7, 9, 10]. Owing to these advantages, SPH has been increasingly applied to industrial problems involving complex, multiphysics scenarios under transient conditions [11–16].

Originally designed for highly compressible flows in astrophysics and shock dynamics [3, 17, 18], early SPH research on fluids was naturally extended to simulate general fluid motion through a weakly compressible approach, which allows slight compressibility in the fluid and is known as the weakly compressible SPH (WCSPH) formulation [8]. However, numerical instabilities in WCSPH formulations, especially under low-Mach-number conditions, were frequently observed in early investigations. Over time, stabilization techniques have been introduced; most notably δ -SPH [19, 20] and low dissipation Riemann-SPH [21, 22] have been introduced to mitigate these issues and improve the robustness and accuracy of SPH under nearly incompressible flow conditions.

As an alternative to weakly compressible formulations, the Incompressible SPH (ISPH) method was developed to simulate truly incompressible flows by solving a pressure Poisson equation (PPE) derived from the continuity equation [23–25]. Rooted in the projection method and Helmholtz decomposition, ISPH decouples velocity and pressure updates, ensuring that the resulting velocity field satisfies the incompressibility constraint. This approach has shown notable accuracy and stability and is now widely adopted in SPH-based transient simulations [23].

While projection-based methods dominate the SPH community, other pressure–velocity coupling strategies such as the SIMPLE (Semi-Implicit Method for Pressure-Linked Equations) [26–29] and PISO (Pressure-Implicit with Splitting of Operators) [30] families have been extensively studied in mesh-based CFD and are widely implemented in commercial solvers [31]. These methods are particularly effective for structured and unstructured meshes and offer advantages in terms of robustness and implementation simplicity. Among them, PISO is typically preferred for unsteady simulations, whereas SIMPLE is better suited for steady-state computations [31–36]. However, these methods have not been thoroughly explored within the SPH framework.

To the best of the author’s knowledge, only a few recent studies have attempted to integrate such segregated algorithms into SPH [37, 38]. A PISO-based SPH formulation has been adopted for incompressible flows, incorporating a predictor–corrector procedure with two pressure-correction steps to enhance the stability and accuracy of the pressure field [37]. Another study introduced a SIMPLE-based SPH approach, in which pressure and viscosity solvers are iteratively coupled to achieve stable performance over a wide viscosity spectrum [38]. Notably, both approaches were developed exclusively for transient simulations.

This reflects a broader trend in SPH research, which has historically focused on unsteady and dynamic phenomena such as wave breaking, sloshing, and multiphase interactions. Consequently, the development of steady-state solvers within the SPH framework has received relatively little attention, despite their potential utility in providing well-converged initial conditions or analyzing equilibrium states in engineering applications.

In conventional CFD practice, however, steady-state solutions are often a prerequisite for reliable unsteady simulations, particularly in problems involving complex geometries, strongly coupled source terms, or nonuniform initial conditions [36, 39, 40]. Developing a robust and efficient steady-state solver within the SPH framework therefore represents a critical step toward broadening the applicability of SPH to a wider class of practical multi-physics problems.

While it is feasible to use steady-state results obtained from conventional mesh-based CFD solvers as initial conditions for transient SPH simulations, this approach requires an intermediate post-processing step to apply the mesh-based solution onto the SPH framework. By contrast, developing a steady-state solver directly within the Eulerian SPH framework enables a seamless transition from steady-state computation to transient analysis, eliminating the need for such post-processing and ensuring consistency across simulation steps. This integration reduces computational costs of SPH simulations where both steady and unsteady behaviors are of interest.

In this study, we propose a steady-state formulation of the SIMPLE algorithm within the Eulerian SPH framework to compute incompressible flow and heat transfer problems. By removing the time-dependent terms from the governing equations, the method directly solves for the steady-state pressure, velocity, and temperature fields using a matrix-based formulation. A matrix-free iterative solver, specifically Bi-CGSTAB, is employed to efficiently handle the resulting sparse linear systems, and the implementation is fully parallelized on GPUs to enhance computational performance. The proposed solver is validated through a series of benchmark problems, demonstrating its accuracy, robustness, and computational efficiency.

The remainder of this paper is structured as follows: Section 2 presents the governing equations and outlines the SIMPLE algorithm adapted for steady-state SPH computations, along with its matrix formulation. Section 3 describes the SPH discretization strategies used to express the momentum, pressure correction, and heat conduction equations within the Eulerian framework. Section 4 details the numerical implementation, including the use of under-relaxation, matrix-free Bi-CGSTAB iteration, and GPU parallelization. Section 5 validates the proposed solver through a series of benchmark simulations, demonstrating its accuracy and computational performance. Finally, Section 6 concludes the study and discusses potential future applications.

2 | Governing Equations and SIMPLE Algorithms

The steady-state solver aims to compute the velocity and pressure fields by solving the incompressible Navier–Stokes equations. Unlike conventional transient SPH methods, which solve the time-dependent form of the governing equations by retaining temporal derivative terms and relying on repeated particle interactions at each time step, the present approach eliminates the time-dependent terms entirely and directly formulates the governing equations in a steady-state form. This reformulation enables a matrix-based solution strategy that targets the final converged solution without the need for time integration, thereby significantly enhancing computational efficiency and scalability for steady-state problems. In this section, the SIMPLE algorithm implemented in this study is described in detail, both in matrix form and in its discretized implementation.

2.1 | Governing Equations

The motion and thermal behavior of a Newtonian, incompressible fluid in the Eulerian framework are governed by the following unsteady equations:

$$\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} = -\frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{g} \quad (1)$$

$$\nabla \cdot \mathbf{u} = 0 \quad (2)$$

$$\frac{\partial T}{\partial t} + \mathbf{u} \cdot \nabla T = \alpha \nabla^2 T + \frac{q}{\rho c_p} \quad (3)$$

where $\mathbf{u}$ is the velocity vector, p is the pressure, T is the temperature, ρ is the density, $\nu = \mu/\rho$ is the kinematic viscosity, $\alpha = k/(\rho c_p)$ is the thermal diffusivity, and q is the volumetric heat source.

In this study, the focus is on the steady-state solution of fluid flow and heat transfer problems.

Therefore, the following assumptions are applied to simplify the governing equations:

- The flow is incompressible and steady state.
- Gravity is included and buoyancy is modeled with the Boussinesq approximation.
- There is no internal heat generation ($q = 0$).
- The momentum and thermal equations are two-way coupled through the buoyancy term so that temperature affects momentum and vice versa.

Applying these assumptions, the governing equations reduce to the following steady-state form:

$$\mathbf{u} \cdot \nabla \mathbf{u} = -\frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{g} + \mathbf{F}_{\text{buoyancy}} \quad (4)$$

$$\mathbf{u} \cdot \nabla T = \alpha \nabla^2 T \quad (5)$$

The momentum/continuity equations (Equations 2 and 4) and the energy equation (Equation 5) are solved as a two-way coupled system. The buoyancy force is modeled by the Boussinesq approximation, $\rho\beta(T - T_{\text{ref}})\mathbf{g}$, so that the temperature field influences the flow, while advection in the energy equation depends on the current velocity field. The two fields are

updated within an outer fixed-point procedure and the process is repeated until both velocity and temperature residuals satisfy prescribed tolerances. When the buoyancy term is neglected, the formulation reduces to a one-way coupling scheme.

2.2 | SIMPLE Algorithm

The SIMPLE is one of the most widely used algorithms for solving the steady incompressible Navier–Stokes equations [26]. It also serves as the foundation for other pressure–velocity coupling methods such as SIMPLER and PISO, which extend the approach to both steady and transient applications. The SIMPLE algorithm resolves the pressure–velocity coupling problem by first computing an intermediate velocity through a predictor step that does not satisfy the continuity condition. This provisional velocity is then substituted into the pressure correction equation, commonly referred to as the PPE to compute an updated pressure field. The velocity field is subsequently corrected using this updated pressure, yielding velocity and pressure fields that satisfy both the momentum and continuity equations.

The algorithm can be implemented in either an incremental form, where corrections are applied to intermediate variables, or a matrix form, where all terms are assembled and solved as linear algebraic systems.

The general steps of the SIMPLE algorithm can be summarized as follows:

1. *Guess the pressure field.* Provide an initial guess for the pressure field, or use the pressure field obtained from the previous iteration.
2. *Solve the momentum equations to obtain the intermediate velocity field.* Compute the provisional velocity field using the momentum equations, based on the pressure gradient derived from the guessed pressure.
3. *Solve the pressure correction equation.* Construct and solve the PPE using the divergence of the intermediate velocity to enforce the continuity condition.
4. *Correct the pressure and velocity fields.* Update the pressure and velocity fields using the computed pressure correction to ensure mass conservation and momentum consistency.

If heat transfer is of interest, an energy equation is solved in a two-way coupled scheme with the momentum and continuity equations. The convective term in the energy equation is evaluated with the instantaneous velocity field, while buoyancy feeds back into the momentum equations under the Boussinesq approximation. Velocity/pressure and temperature are updated within a single outer iteration until two residual tolerances are met, enabling natural convection simulations.

2.3 | Matrix Formulation of the SIMPLE Algorithm

In the present study, Step 2 of the SIMPLE algorithm is simplified by approximating the intermediate velocity using the velocity from the previous iteration. This approach avoids solving a separate linear system for the velocity at each iteration, thereby reducing computational cost. While the governing equations are expressed in matrix form to enhance numerical stability, the overall scheme retains a semi-implicit nature due to the sequential treatment of pressure and velocity.

The flowchart of the SIMPLE algorithm is presented in Figure 1. At each iteration k , (i) the momentum system is assembled including the buoyancy term $\rho\beta(T - T_{\text{ref}})\mathbf{g}$; (ii) one SIMPLE cycle is performed to obtain $\mathbf{u}^{k+1}, p^{k+1}$; (iii) with $\mathbf{u}^{k+1}$ the energy equation is solved to update T^{k+1} . The outer loop proceeds until both the flow and thermal residuals meet prescribed tolerances. This procedure enables two-way coupling between momentum/continuity and energy.

2.3.1 | Pressure Correction Equation

In Equation (4), gravity is absorbed into the pressure gradient by introducing the piezometric pressure $\Phi = P - \rho\mathbf{g} \cdot \mathbf{x}$ so that $-\nabla P + \rho\mathbf{g} = -\nabla\Phi$. The remaining buoyancy force becomes $\rho\beta(T - T_{\text{ref}})\mathbf{g}$.

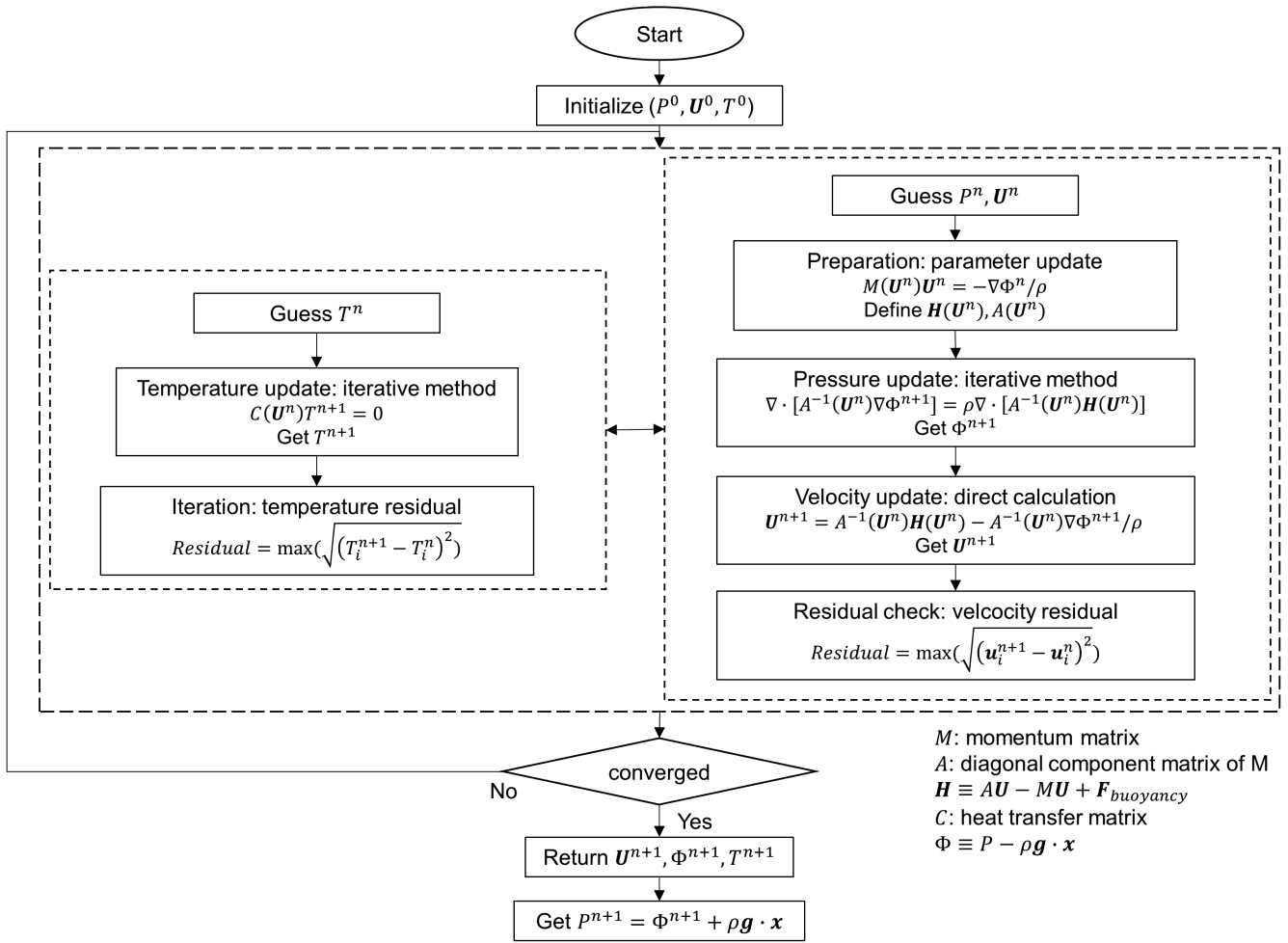


FIGURE 1 | Flowchart illustrating the solution procedure of the implicit steady solver.

The key step in the SIMPLE algorithm is the construction of the pressure correction equation. The momentum equation, as shown in Equation (4), can be rearranged to isolate the velocity-related terms and expressed in operator form as:

$$MU = -\frac{\nabla\Phi}{\rho} + \beta(T - T_{\text{ref}})g \quad (6)$$

Here, M denotes the momentum operator, which includes both the nonlinear convective term and the diffusive term arising from viscous effects.

To derive the PPE, an auxiliary term H is introduced, defined as:

$$H \equiv AU - MU - \beta(T - T_{\text{ref}})g \quad (7)$$

where A is the diagonal matrix.

From the definition of H , Equation (6) can be rearranged into the form of Equation (8). Equation (9) is derived by applying the inverse of A to both sides of Equation (8).

$$AU - H = -\frac{\nabla\Phi}{\rho} \quad (8)$$

$$U - A^{-1}H = -A^{-1}\frac{\nabla\Phi}{\rho} \quad (9)$$

Equation (10) is derived by applying the divergence operator to both sides of Equation (9). Imposing the incompressible condition ($\nabla \cdot \mathbf{U} = 0$) yields the final form of PPE, as shown in Equation (11).

$$\nabla \cdot \mathbf{U} - \nabla \cdot (\mathbf{A}^{-1} \mathbf{H}) = -\nabla \cdot \left(\mathbf{A}^{-1} \frac{\nabla \Phi}{\rho} \right) \quad (10)$$

$$\nabla \cdot \left(\mathbf{A}^{-1} \frac{\nabla \Phi}{\rho} \right) = \nabla \cdot (\mathbf{A}^{-1} \mathbf{H}) \quad (11)$$

The physical pressure can be recovered from $P = \Phi + \rho \mathbf{g} \cdot \mathbf{x}$.

2.3.2 | Velocity Correction

By substituting the pressure obtained from Equation (11) into Equation (9), the velocity field can be determined, leading to Equation (12).

$$\mathbf{U} = \mathbf{A}^{-1} \mathbf{H} - \mathbf{A}^{-1} \frac{\nabla \Phi}{\rho} \quad (12)$$

2.3.3 | Heat Conduction

Assuming the absence of external heat sources, the heat conduction equation can similarly be expressed using the operator $\mathbf{C}$, as shown in Equation (13). Here, $\mathbf{C}$ encompasses both the convection term induced by the fluid flow and the heat conduction term. The operator $\mathbf{C}$ is dependent on the specific heat, thermal conductivity, and density, as well as the flow velocity. Since the operator does not explicitly depend on the temperature, there are no nonlinear terms, which allows for a significantly simpler solution of the temperature field compared to the momentum equation.

$$\mathbf{C}T = 0 \quad (13)$$

When an initial condition that is very close to the exact solution of the problem is provided, the steady-state solution can be obtained with just a single computation. However, in most cases, the exact solution is unknown, and thus multiple iterations are required to reach the solution. In the equations above, the difference between the velocity/temperature field computed at the current step and the velocity/temperature field from the previous step is set as the residual. The iteration loop is terminated when the residual falls below a specified criterion; otherwise, the calculations are iteratively performed based on the aforementioned equations until convergence is achieved.

3 | SPH Discretization of Steady Solver

In this section, the basic SPH approximations are introduced. The SPH-based steady-state solver, formulated using the SIMPLE algorithm, is discretized and described in detail.

3.1 | SPH Approximation

In the SPH formulation, the function $f(\mathbf{r})$ at the position $\mathbf{r}$ can be approximated by integrating over the domain Ω using the smoothing kernel function $W(\mathbf{r} - \mathbf{r}', h)$, as shown in Equation (14), where h is the smoothing length [2].

$$f(\mathbf{r}) = \int_{\Omega} f(\mathbf{r}') W(\mathbf{r} - \mathbf{r}', h) d\mathbf{r}' \quad (14)$$

In this study, the Wendland 2nd kernel is used as the smoothing kernel function, which is defined by Equations (15) and (16) [41]. The smoothing length is set to $h = 1.5dx$, where dx is the particle spacing.

$$W_{ij}(r_{ij}, h) = \frac{7}{4\pi h^2} \left(1 - \frac{r_{ij}}{2h}\right)^4 \left(1 + 4\frac{r_{ij}}{2h}\right) \text{ for 2D} \quad (15)$$

$$W_{ij}(r_{ij}, h) = \frac{21}{16\pi h^3} \left(1 - \frac{r_{ij}}{2h}\right)^4 \left(1 + 4\frac{r_{ij}}{2h}\right) \text{ for 3D} \quad (16)$$

Upon discretization, the fluid is represented by individual particles, and the function $f(\mathbf{r}_i)$ at the position of i particle is expressed as shown in Equation (17):

$$f(\mathbf{r}_i) = \sum_j f(\mathbf{r}_j) W_{ij}(\mathbf{r}_{ij}, h) \frac{m_j}{\rho_j}, \quad (17)$$

where the subscript j denotes neighboring particles, the physical quantities associated with the i particle are defined based on the physical quantities of the neighboring particles.

The SPH gradient, divergence, and Laplacian operators are expressed in the forms, as shown in Equations (18–20) [42, 43].

$$\nabla f(\mathbf{r}_i) = \sum_j \frac{m_j}{\rho_j} \{f(\mathbf{r}_j) \pm f(\mathbf{r}_i)\} \nabla W_{ij} \quad (18)$$

$$\nabla \cdot f(\mathbf{r}_i) = \sum_j \frac{m_j}{\rho_j} \{f(\mathbf{r}_j) \pm f(\mathbf{r}_i)\} \cdot \nabla W_{ij} \quad (19)$$

$$\nabla^2 f(\mathbf{r}_i) = \sum_j 2 \frac{m_j}{\rho_j} \{f(\mathbf{r}_i) - f(\mathbf{r}_j)\} \frac{\mathbf{r}_{ij} \cdot \nabla W_{ij}}{|\mathbf{r}_{ij}|^2} \quad (20)$$

3.2 | Eulerian SPH Discretization of Navier–Stokes Equation

This study analyzes fluid flow within a stationary Eulerian framework, where the convection term associated with the flow velocity is retained in the governing equations. As described in Equation (4), the momentum equation governs the steady-state behavior of an incompressible Newtonian fluid. For the purpose of discretization, the momentum equation is revisited at the n -th outer iteration step in the SIMPLE algorithm, where the velocity $\mathbf{u}^n$ from the previous step is used in evaluating the convective and diffusive terms.

$$(\mathbf{u}^n \cdot \nabla) \mathbf{u}^n = -\frac{\nabla \Phi^{n+1}}{\rho} + \frac{\mu}{\rho} \nabla^2 \mathbf{u}^n + \beta(T^{n+1} - T_{\text{ref}}) \mathbf{g} \quad (21)$$

The SPH discretization of the convection term can be formulated in various ways; a numerically stable tensor product form is adopted in this study [44]. The SPH discretized form of the Navier–Stokes equation is as shown in Equation (22), where $\bar{\mu}_{ij} = \frac{2\mu_i\mu_j}{\mu_i+\mu_j}$.

$$\mathbf{u}_i^n \cdot \sum_j \frac{m_j}{\rho_j} (\mathbf{u}_j^n - \mathbf{u}_i^n) \otimes \nabla_i W_{ij} = -\frac{1}{\rho_i} \sum_j \frac{m_j}{\rho_j} (\Phi_i^{n+1} + \Phi_j^{n+1}) \nabla_i W_{ij} + \frac{1}{\rho_i} \sum_j 2\bar{\mu}_{ij} \frac{m_j}{\rho_j} (\mathbf{u}_i^n - \mathbf{u}_j^n) \frac{\mathbf{r}_{ij} \cdot \nabla_i W_{ij}}{|\mathbf{r}_{ij}|^2} + \beta(T^{n+1} - T_{\text{ref}}) \mathbf{g} \quad (22)$$

Although Equation (22) is not solved directly in this study, it is critical for constructing the linear system: $\mathbf{M}\mathbf{U} = -\nabla\Phi/\rho + \beta(T - T_{\text{ref}})\mathbf{g}$, which is required for the SIMPLE algorithm. To this end, each term in Equation (22) is reorganized into a matrix–vector product form. The SPH-based matrix representation of the momentum equation is:

$$\begin{aligned} & \mathbf{u}_i^n \sum_j \frac{m_j}{\rho_j} \left(-\mathbf{u}_i^n \cdot \nabla_i W_{ij} - \frac{2\bar{\mu}_{ij}}{\rho_i} \frac{\mathbf{r}_{ij} \cdot \nabla_i W_{ij}}{|\mathbf{r}_{ij}|^2} \right) + \sum_j \frac{m_j}{\rho_j} \mathbf{u}_j^n \left(\mathbf{u}_i^n \cdot \nabla_i W_{ij} + \frac{2\bar{\mu}_{ij}}{\rho_i} \frac{\mathbf{r}_{ij} \cdot \nabla_i W_{ij}}{|\mathbf{r}_{ij}|^2} \right) \\ &= -\frac{1}{\rho_i} \sum_j \frac{m_j}{\rho_j} (\Phi_i^{n+1} + \Phi_j^{n+1}) \nabla_i W_{ij} + \beta(T^{n+1} - T_{\text{ref}}) \mathbf{g} \end{aligned} \quad (23)$$

This leads to the definition of the momentum matrix $\mathbf{M} = \{M_{i,j}\}$, where:

$$M_{ij} \equiv \begin{cases} \sum_j \frac{m_j}{\rho_j} \left(-\mathbf{u}_i^n \cdot \nabla_i W_{ij} - \frac{2\bar{\mu}_{ij}}{\rho_i} \frac{\mathbf{r}_{ij} \cdot \nabla_i W_{ij}}{|\mathbf{r}_{ij}|^2} \right), & (i = j) \\ \frac{m_j}{\rho_j} \left(\mathbf{u}_i^n \cdot \nabla_i W_{ij} + \frac{2\bar{\mu}_{ij}}{\rho_i} \frac{\mathbf{r}_{ij} \cdot \nabla_i W_{ij}}{|\mathbf{r}_{ij}|^2} \right), & (i \neq j) \end{cases} \quad (24)$$

This formulation explicitly defines the components of the matrix $\mathbf{M}$, that appear in the term $\mathbf{M}\mathbf{U} = -\nabla\Phi/\rho + \beta(T - T_{\text{ref}})\mathbf{g}$, which forms the foundation for deriving the PPE in the next step of the SIMPLE algorithm.

From the constructed momentum matrix, the diagonal matrix $\mathbf{A}$ and the auxiliary term $\mathbf{H}$ are defined in preparation for solving the PPE. The diagonal matrix $\mathbf{A} = \{A_{i,j}\}$, is constructed by retaining only the diagonal terms of the momentum matrix $\mathbf{M}$:

$$A_{ij} = \begin{cases} \sum_j \frac{m_j}{\rho_j} \left(-\mathbf{u}_i^n \cdot \nabla_i W_{ij} - \frac{2\bar{\mu}_{ij}}{\rho_i} \frac{\mathbf{r}_{ij} \cdot \nabla_i W_{ij}}{|\mathbf{r}_{ij}|^2} \right), & (i = j) \\ 0, & (i \neq j) \end{cases} \quad (25)$$

Based on the definition of $\mathbf{H} = \{H_{i,j}\}$, in Equation (7), the auxiliary term can be readily evaluated once $\mathbf{A}$ and $\mathbf{M}$ are constructed.

Since the matrix $\mathbf{A}$ consists only of diagonal components, its inverse can be easily computed. The PPE is implicitly solved by expressing Equation (11) in the matrix form $LP = \mathbf{B}$, which is SPH discretized as shown in Equations (26–28).

$$\sum_j L_{ij} (\Phi_i^{n+1} - \Phi_j^{n+1}) = B_i \quad (26)$$

$$L_{ij} = \frac{1}{A_{ij}^{-1}} \frac{m_j}{\rho_j} \frac{2\mathbf{r}_{ij} \cdot \nabla W_{ij}}{|\mathbf{r}_{ij}|^2} = \frac{2A_{ii}^{-1}A_{jj}^{-1}}{A_{ii}^{-1} + A_{jj}^{-1}} \frac{m_j}{\rho_j} \frac{2\mathbf{r}_{ij} \cdot \nabla W_{ij}}{|\mathbf{r}_{ij}|^2} \quad (27)$$

$$B_i = \rho_i \nabla \cdot (A_{ii}^{-1} \mathbf{H}_i) \quad (28)$$

The inverse matrix $\mathbf{A}^{-1}$ and $\mathbf{H}$ obtained from Equations (25) and (7), respectively, along with the pressure field obtained from Equation (26), are substituted into Equation (11) to compute the velocity $\mathbf{U}^{n+1}$ for the next step. Then, for all particles, the difference between the velocities of the current and next step is calculated, and the maximum value is set as the residual. The iteration loop terminates when this residual becomes lower than the predefined threshold.

$$\text{Residual}_U = \max(\text{abs}(\mathbf{u}_i^{n+1} - \mathbf{u}_i^n)) < 1e - 6 \text{ [m/s]} \quad (29)$$

3.3 | Eulerian SPH Discretization of Heat Conduction Equation

As described in Equation (5), the energy equation governs the heat transfer without external heat sources term. At the n -th outer iteration, the steady energy equation is solved for T^{n+1} with the convective term evaluated using the previous velocity field $\mathbf{u}^n$.

$$c_p(\mathbf{u}^n \cdot \nabla)T^{n+1} = \frac{k}{\rho} \nabla^2 T^{n+1} \quad (30)$$

The SPH discretization form of heat conduction equation is as shown in Equation (31), where $\bar{k}_{ij} = \frac{2k_i k_j}{k_i + k_j}$.

$$c_p \mathbf{u}_i^n \cdot \sum_j \frac{m_j}{\rho_j} (T_j^{n+1} - T_i^{n+1}) \nabla_i W_{ij} = \frac{1}{\rho_i} \sum_j \frac{2\bar{k}_{ij}}{\rho_i} \frac{m_j}{\rho_j} (T_i^{n+1} - T_j^{n+1}) \frac{\mathbf{r}_{ij} \cdot \nabla_i W_{ij}}{|\mathbf{r}_{ij}|^2} \quad (31)$$

Following a similar formulation as in the momentum equation, the temperature equation is reorganized into matrix form to enable linear system solution.

$$T_i^{n+1} \sum_j \frac{m_j}{\rho_j} \left(-c_p \mathbf{u}_i^n \cdot \nabla_i W_{ij} - \frac{2\bar{k}_{ij}}{\rho_i} \frac{\mathbf{r}_{ij} \cdot \nabla_i W_{ij}}{|\mathbf{r}_{ij}|^2} \right) + \sum_j \frac{m_j}{\rho_j} T_j^{n+1} \left(c_p \mathbf{u}_i^n \cdot \nabla_i W_{ij} + \frac{2\bar{k}_{ij}}{\rho_i} \frac{\mathbf{r}_{ij} \cdot \nabla_i W_{ij}}{|\mathbf{r}_{ij}|^2} \right) \quad (32)$$

This expression leads to the definition of the heat transfer coefficient matrix $\mathbf{C} = \{C_{ij}\}$, where:

$$C_{ij} = \begin{cases} \sum_j \frac{m_j}{\rho_j} \left(-c_p \mathbf{u}_i^n \cdot \nabla_i W_{ij} - \frac{2\bar{k}_{ij}}{\rho_i} \frac{\mathbf{r}_{ij} \cdot \nabla_i W_{ij}}{|\mathbf{r}_{ij}|^2} \right), & (i = j) \\ \frac{m_j}{\rho_j} \left(c_p \mathbf{u}_i^n \cdot \nabla_i W_{ij} + \frac{2\bar{k}_{ij}}{\rho_i} \frac{\mathbf{r}_{ij} \cdot \nabla_i W_{ij}}{|\mathbf{r}_{ij}|^2} \right), & (i \neq j) \end{cases} \quad (33)$$

The resulting linear system $\mathbf{C}\mathbf{T}^{n+1} = \mathbf{0}$ is solved for the temperature field using the previously velocity $\mathbf{u}^n$. For the temperature field, the difference between the temperatures of the current and next step is calculated, and the maximum value is set as the residual. The iteration loop terminates when this residual becomes lower than the predefined threshold.

$$\text{Residual}_T = \max(\text{abs}(T_i^{n+1} - T_i^n)) < 1e - 6 \text{ [K]} \quad (34)$$

3.4 | Boundary Conditions

The matrix obtained by transforming the Navier–Stokes equations represents the governing equations for the fluid, and thus cannot be applied to wall particles. The pressure for wall particles is determined by the generalized boundary condition, and since gravity is neglected, the equation simplifies to the form shown in Equation (35) [45].

$$\Phi_{\text{wall}} = \frac{\sum_{j \in \text{fluid}} \Phi_j W_{ij}}{\sum_{j \in \text{fluid}} W_{ij}} \quad (35)$$

Wall particles do not participate in the velocity update calculations; however, a fictitious velocity is assigned to enforce the no-slip condition. This is done with the goal of making the velocity of fluid particles adjacent to the wall equal to zero. The wall velocity is assigned as shown in Equation (36) [45], where U_0 is the prescribed wall velocity.

$$\mathbf{U}_{\text{wall}} = 2\mathbf{U}_0 - \frac{\sum_{j \in \text{fluid}} \mathbf{U}_j W_{ij}}{\sum_{j \in \text{fluid}} W_{ij}} \quad (36)$$

Similarly, the boundary condition for the temperature can also be applied as shown in Equation (37), where T_0 is the prescribed wall temperature.

$$T_{\text{wall}} = 2T_0 - \frac{\sum_{j \in \text{fluid}} T_j W_{ij}}{\sum_{j \in \text{fluid}} W_{ij}} \quad (37)$$

4 | Implementation

4.1 | Under-Relaxation

When executing the steady solver, pressure oscillations may occur depending on the case. These oscillations not only unnecessarily increase the number of iterations in the Bi-CGSTAB method but also hinder the reduction of the residual in the outer iteration, which is defined by the difference in velocity between the current and next step. To reduce pressure oscillations and improve convergence, under-relaxation is introduced, which is used to update the pressure for the next step.

The under-relaxation factor α_p is typically set to 0.2, but the problems where pressure oscillations are small, a large value can be used. Conversely, for the problems with significant pressure oscillations, a smaller value is applied. This is expressed as shown in Equation (38). Here, Φ_{new}^{n+1} is the pressure for the next step after applying under-relaxation, while Φ^{n+1} is the pressure for the next step computed by solving the PPE implicitly.

$$\Phi_{\text{new}}^{n+1} = \alpha_p \Phi^{n+1} + (1 - \alpha_p) \Phi^n \quad (38)$$

4.2 | Iterative Method: Matrix-Free Bi-CGSTAB

The derived PPE and heat transfer equation are solved implicitly using an iterative method. An iterative method refers to a numerical technique that starts from an initial guess and progressively updates the solution through repeated steps, converging toward an accurate approximation. These methods are particularly advantageous for solving large sparse systems and are more efficient and scalable than direct methods, especially when applied to parallel computing environments.

Iterative methods are broadly classified into stationary and nonstationary methods. Stationary methods like Jacobi, Gauss–Seidel, and SOR use fixed iteration formulas and are easy to implement but converge slowly, especially for large or ill-conditioned systems. Nonstationary methods, often based on the Krylov subspace approach [46], adapt their strategy at each step and offer faster convergence for large, sparse, and nonsymmetric linear systems.

In this study, the Bi-Conjugate Gradient Stabilized (Bi-CGSTAB) method is adopted. Bi-CGSTAB improves the convergence stability of the original Bi-Conjugate Gradient (BiCG) method by suppressing the oscillatory behavior of the residual, enabling faster and more robust convergence. It is also well-suited for nonsymmetric matrices, making it highly appropriate for the proposed formulation [47].

The Bi-CGSTAB algorithm constructs search directions within the Krylov subspace by using the residual vector and enforcing orthogonality conditions. During each iteration, scalar parameters such as α , β , and ω are computed to update the solution vector and minimize the residual. The overall procedure of the algorithm is illustrated in Figure 2.

The inner Bi-CGSTAB tolerances for the pressure Poisson and energy equations are adjusted adaptively based on the outer coupled residual. Specially, with $R_{\text{out}} = \max(\text{Residual}_U, \text{Residual}_T)$, the solver sets relative residual criterions $\tau_p^2 = \max(10^{-8}, 0.1R_{\text{out}})$ and $\tau_T^2 = \max(10^{-8}, 0.1R_{\text{out}})$. This method starts with a loose residual criterion (10^{-3}) and tightens to 10^{-8} as the coupled iteration converges, avoiding unnecessary calculation in early iterations while preserving the final accuracy.

```

function Preconditioned BICGSTAB ( $A, b, x_0$ )
 $r_0 := b - Ax_0$ ;  $p_0 := r_0$ ;
 $\hat{r}_0$  is an arbitrary vector which satisfies  $(\hat{r}_0, r_0) \neq 0$ ,
e.g.,  $\hat{r}_0 = r_0$ ;
 $i = 0$ 
while( $\|r_i\| < \tau^2$ )
    Computation  $\hat{p}_i := M^{-1}p_i$ 
    Computation  $s_i := A\hat{p}_i$ 
    Reduction  $(r_0, r_i)$ ;  $(r_0, s_i)$ 
     $\alpha_i := (r_0, r_i)/(r_0, s_i)$ 
     $q_i := r_i - \alpha_i s_i$ 
    Computation  $\hat{q}_i := M^{-1}q_i$ 
    Computation  $y_i := A\hat{q}_i$ 
    Reduction  $(q_i, y_i)$ ;  $(y_i, y_i)$ 
     $\omega_i := (q_i, y_i)/(y_i, y_i)$ 
     $x_{i+1} := x_i + \alpha_i \hat{p}_i + \omega_i \hat{q}_i$ 
     $r_{i+1} := q_i - \omega_i y_i$ 
    Reduction  $(r_0, r_{i+1})$ 
     $\beta_i := (\alpha_i/\omega_i)(r_0, r_{i+1})/(r_0, r_i)$ 
     $p_{i+1} := r_{i+1} + \beta_i(p_i - \omega_i s_i)$ 
end function

```

FIGURE 2 | The overall procedure of Bi-CGSTAB algorithm.

To enhance computational efficiency, a matrix-free implementation of Bi-CGSTAB is employed in this study. Instead of storing the $n \times n$ coefficient matrix $\mathbf{A}$ in memory, the matrix–vector products $\mathbf{A}\mathbf{p}$, $\mathbf{A}\mathbf{q}$ are computed directly. This approach significantly reduces memory requirements and is particularly effective for solving sparse systems. In addition, it is well-suited to parallel implementation on GPU architectures.

In summary, the PPE and heat transfer equation in this study are solved using a matrix-free Bi-CGSTAB method, which ensures numerical stability, memory efficiency, and high computational performance.

4.3 | GPU Implementation

The SPH methodology requires a significant computational workload, resulting in high computational costs. However, these operations can be parallelized, and implementing a GPU-based algorithm enables highly efficient computations. In this study, the GPU-parallelized algorithm is implemented following the methodology of Jo [48], which demonstrates substantially enhanced computational efficiency over CPU-based codes.

The CPU is solely responsible for input and output operations, while all calculations are parallelized and executed on the GPU. The hardware specifications used are an Intel Xeon Gold 6342 CPU@2.80GHz and an NVIDIA A100 80GB PCIe.

In Lagrangian-based SPH methodologies, the most computationally expensive process is the nearest neighbor particle search (NNPS). However, since the present study employs an Eulerian framework with stationary particles, NNPS is required only in the initial computation step, significantly reducing computational time [49]. Similarly, the EISPH method [50], used for quantitative validation, also adopts an Eulerian SPH approach, thereby eliminating the computational burden associated with NNPS.

5 | Benchmark Simulations

In this section, various benchmark problems are conducted to validate the proposed Eulerian SPH steady solver. The benchmark cases include hydrodynamic problems with steady-state solutions: (1) 2D pipe flow without obstacles, (2) 2D pipe flow with obstacles, (3) 2D lid-driven flow, and (4) 2D lid-driven flow with heat transfer (no gravitational force), (5) 2D natural convection flow, (6) 3D lid-driven flow. Cases (4) and (5) are computed with the two-way coupled formulation, in which the momentum/continuity equations and the energy equation are updated within a single outer fixed-point loop. The convergence criterion for terminating the iteration is defined such that the maximum velocity/temperature difference between the current and previous steps for all particles falls below 10^{-6} m/s and 10^{-6} K. The same criterion is also applied to determine the termination point in the transient-based EISPH. The number of particles used and the key characteristics of each benchmark case are summarized in Table 1.

TABLE 1 | Brief information of benchmark simulation.

	Benchmark	Resolution	Characteristics	References
5.1	2D pipe flow (no obstacle)	Coarse ($D/\Delta x = 25$) Middle ($D/\Delta x = 50$) Fine ($D/\Delta x = 100$)	Compared with analytic solution	—
5.2	2D pipe flow with obstacles	$L/\Delta x = 356$	Compared with EISPH	[51]
5.3	2D lid driven flow	$L/\Delta x = 300$	Compared with FDM, ANSYS and EISPH	[36, 50, 52]
5.4	2D lid driven flow with heat transfer (no gravitational force)	$L/\Delta x = 300$	Compared with ANSYS and EISPH	[36, 50]
5.5	2D natural convection flow	$L/\Delta x = 300$	Compared with ANSYS.	[36]
5.6	3D lid driven flow	$L/\Delta x = 150$	Compared with FDM	[52, 53]

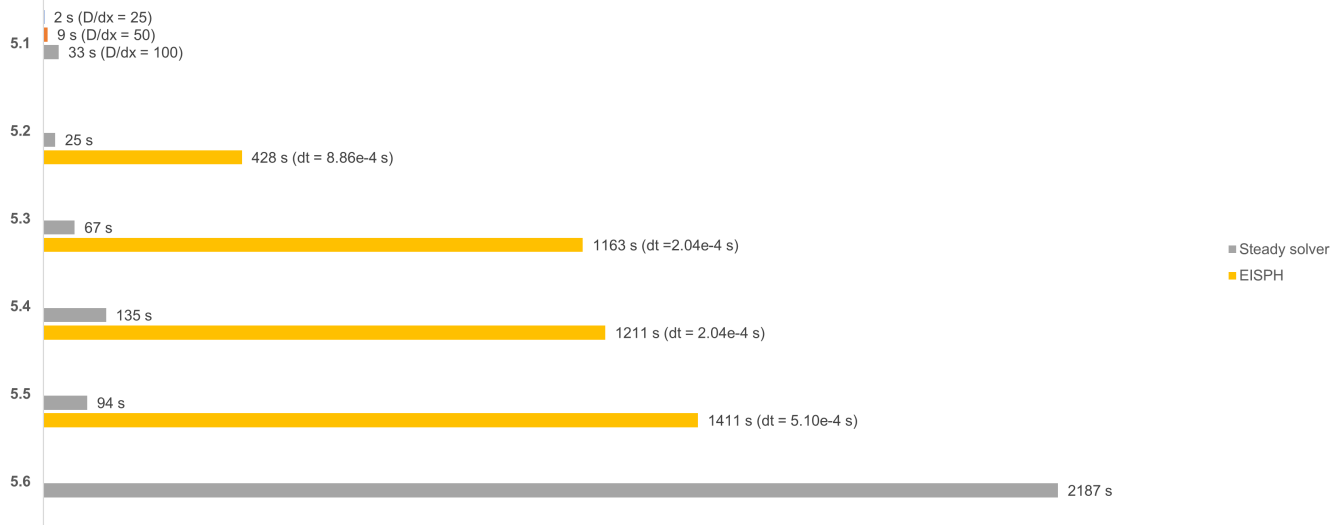


FIGURE 3 | Computation time to reach steady state for each benchmark problem.

The time step in the EISPH method is determined based on stability constraints that do not include the sound speed in the denominator, allowing significantly larger time steps than the WCSPH method.

$$\Delta t_{\text{EISPH}} \leq \left(\text{CFL} \min \left(\frac{h}{||\mathbf{u}_i||} \right), 0.125 \frac{h^2}{\nu} \right) \quad (39)$$

The EISPH method achieves high computation efficiency by solving the PPE only once per time step, rather than iteratively as in conventional ISPH formulations. This approach significantly reduces computational cost while still maintaining sufficient accuracy. Nevertheless, the developed steady-state solver reaches convergence much faster, even when compared to the EISPH, demonstrating its superior computational performance in steady-state simulations.

The computation time for each benchmark problem is summarized in Figure 3. In Section 5.1, the computation time of the steady solver is presented with respect to different spatial resolutions. For other benchmark cases (Sections 5.2, 5.3, 5.4, and 5.5), the computation time is compared under identical conditions between the steady solver and the transient based EISPH (except 3D lid driven because of high computation time).

5.1 | 2D Pipe Flow (No Obstacle)

A 2-dimension pipe flow without obstacles is conducted as a benchmark problem (Figures 4–8). To implement the inflow and outflow conditions, the open boundary conditions proposed by Tafuni et al. [54] are applied. No-slip conditions and generalized pressure boundary conditions are imposed on the wall boundaries. The Reynolds number is defined as $Re = \rho U D / \mu$, where U is the mean flow velocity, D is the pipe width. In this simulation, $U = 0.1$ m/s, $D = 0.5$ m, and $\mu = 10$ Pa · s are used, resulting in $Re = 10$. Since an analytical solution exists for this problem, the steady solver is validated independently without comparison to the EISPH method. A relaxation factor $\alpha_p = 0.3$ is employed. Simulations are conducted at three different resolutions ($D/dx = 25, 50, 100$), and the computational times required to reach a steady state are 2, 9, and 38 s, respectively.

Figure 4 presents the steady-state simulation results: (a) velocity magnitude, (b) x-direction velocity, (c) y-direction velocity, and (d) pressure distribution. The presence of nonzero y-direction velocity near the inlet region, specifically around the entrance length $L = 0.06 \cdot Re \cdot D = 0.3$ m, confirms the expected development of entrance effects. In addition, the no-slip boundary condition is properly enforced, as evidenced by the zero velocity along the channel walls.

Figure 5 compares the velocity profiles in the fully developed region against the analytical solution for various spatial resolutions. As the resolution increases, the computed maximum velocity approaches the theoretical value of $U_{\text{max}} = 0.15$ m/s, indicating improved agreement with the analytical solution.

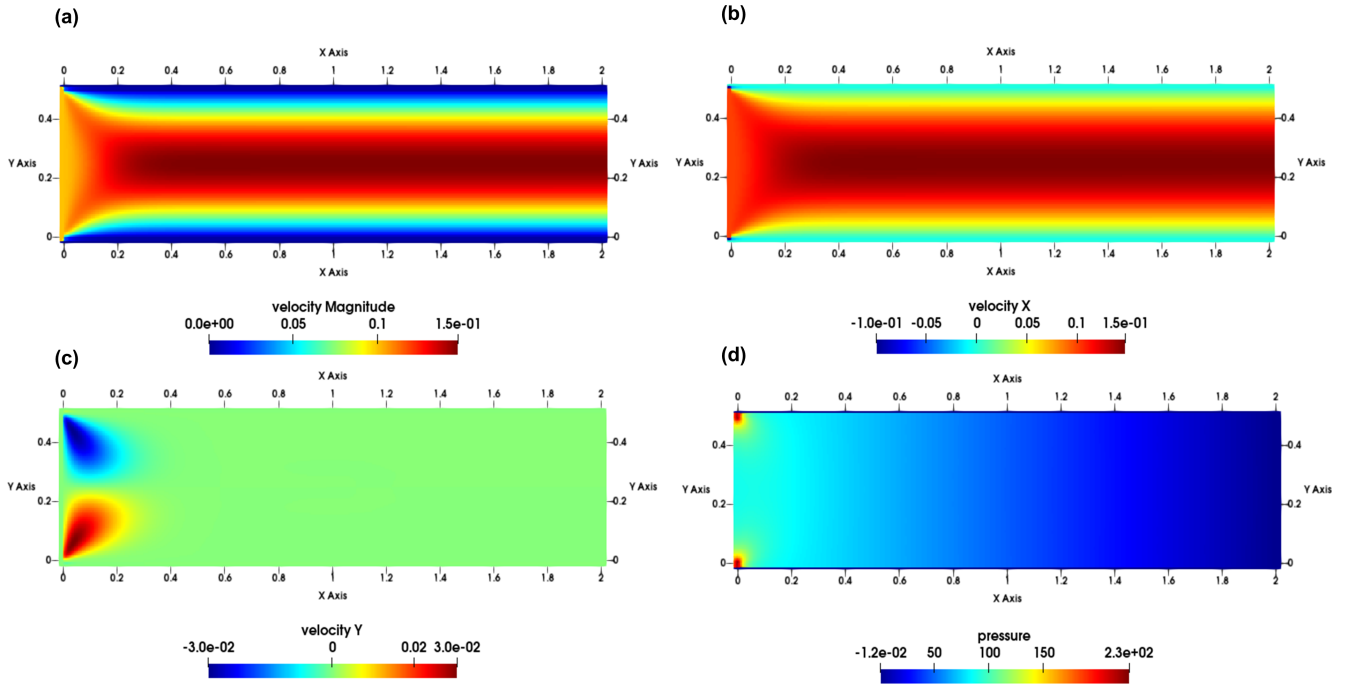


FIGURE 4 | Steady state results for 2D pipe flow: (a) velocity magnitude, (b) x-direction velocity, (c) y-direction velocity, (d) pressure.

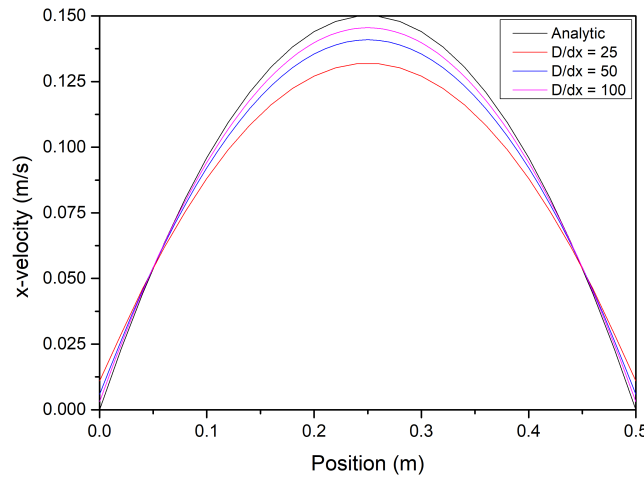


FIGURE 5 | The velocity profiles in the fully developed region for different resolutions with the analytical solution.

Figure 6 shows the L_2 norm of the velocity error as a function of particle resolution. The observed slope of approximately -1 suggests that the method achieves first-order spatial accuracy.

Figure 7 presents the pressure gradient in the fully developed region for different resolutions. The analytical pressure gradient, derived from the parabolic velocity profile $u(y) = 2.4(0.5y - y^2)$, is $\partial p / \partial x = -4.8\mu = -48 \text{ Pa/m}$. The numerical results converge toward this value as the spatial resolution is refined, demonstrating consistency with theory.

Figure 8 shows the L_2 norm of the pressure error versus resolution. Although minor fluctuations are observed, the slope generally trends toward -1 , further confirming first-order convergence in pressure.

5.2 | 2D Pipe Flow With Obstacles

In the previous section, it was demonstrated that the proposed steady-state solver can accurately compute velocity and pressure fields in simple geometries, such as pipe flow without internal obstructions. This section extends the analysis to a more complex two-dimensional pipe flow with multiple internal obstacles, as illustrated in Figures 9–13.

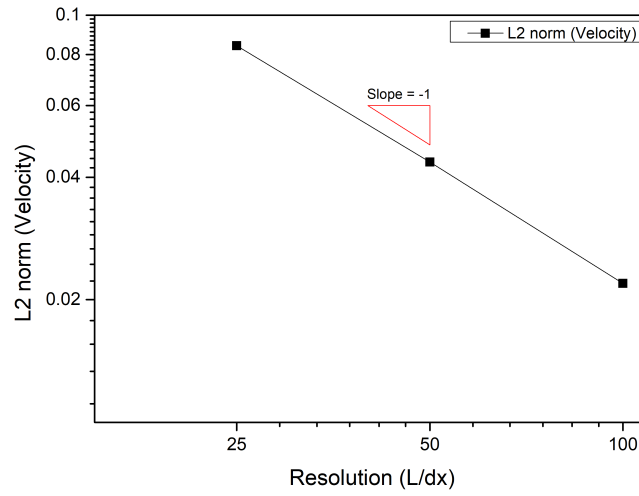


FIGURE 6 | The L2 norm of velocity for different resolutions.

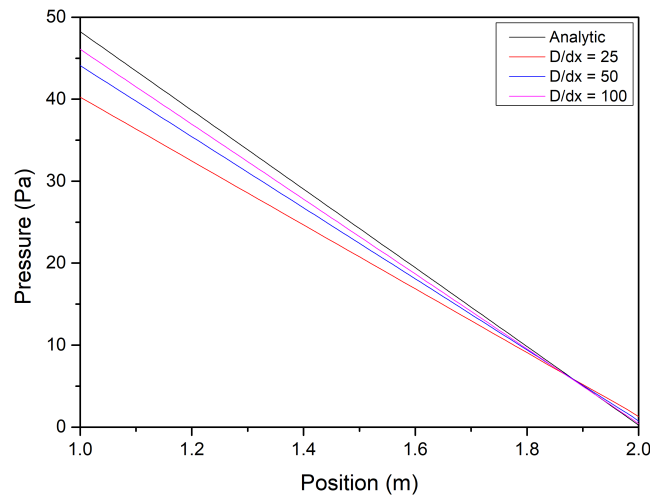


FIGURE 7 | The pressure drop in the fully developed region for different resolutions with the analytical resolutions.

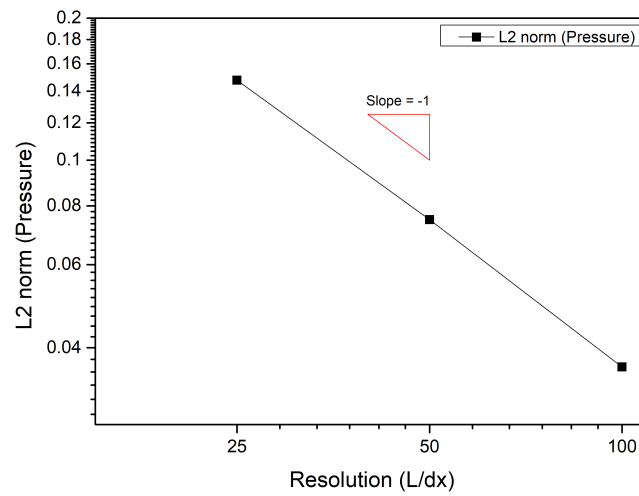


FIGURE 8 | The L2 norm of pressure for different resolutions.

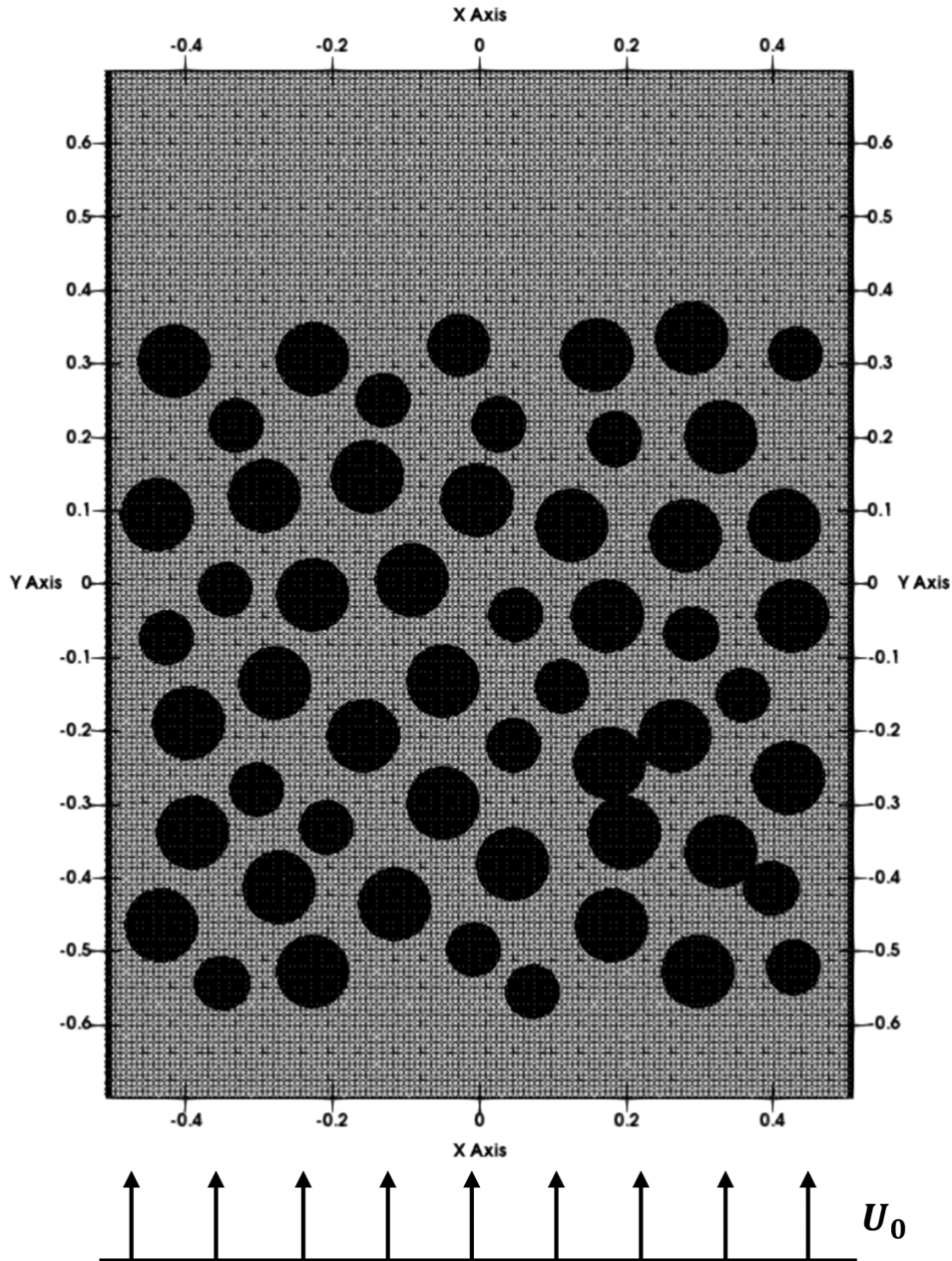


FIGURE 9 | Computational domain for the 2D pipe flow with obstacles.

Figure 9 shows the computational domain, which includes several circular obstacles of varying sizes embedded within a straight 2D pipe. A uniform inlet velocity is applied in the +y-direction. The domain has a length of 1.381 m and a width of 1 m, discretized using a particle spacing of 0.0028 m. The computational domain consists of 174,084 particles with a spatial resolution of $D/\Delta x = 356$.

Boundary conditions applied at the walls include the no-slip condition and a generalized pressure boundary condition at the outlet. The Reynolds number is set to $Re = \rho U D / \mu = 40$, where $U = 2 \times 10^{-3}$ m/s, $D = 1$ m, and $\mu = 5 \times 10^{-2}$ Pa · s. The proposed steady-state solver reaches convergence in 25 s, whereas the transient-based EISPH solver requires 428 s under a CFL number of 0.5 and a time step size of $\Delta t = 8.86 \times 10^{-4}$ s.

Figure 10 presents the steady-state simulation results for (a) velocity magnitude, (b) x-direction velocity, (c) y-direction velocity, and (d) pressure distribution. As expected, higher velocities are observed in the narrow flow channels, accompanied by a well-developed pressure gradient across the porous structure.

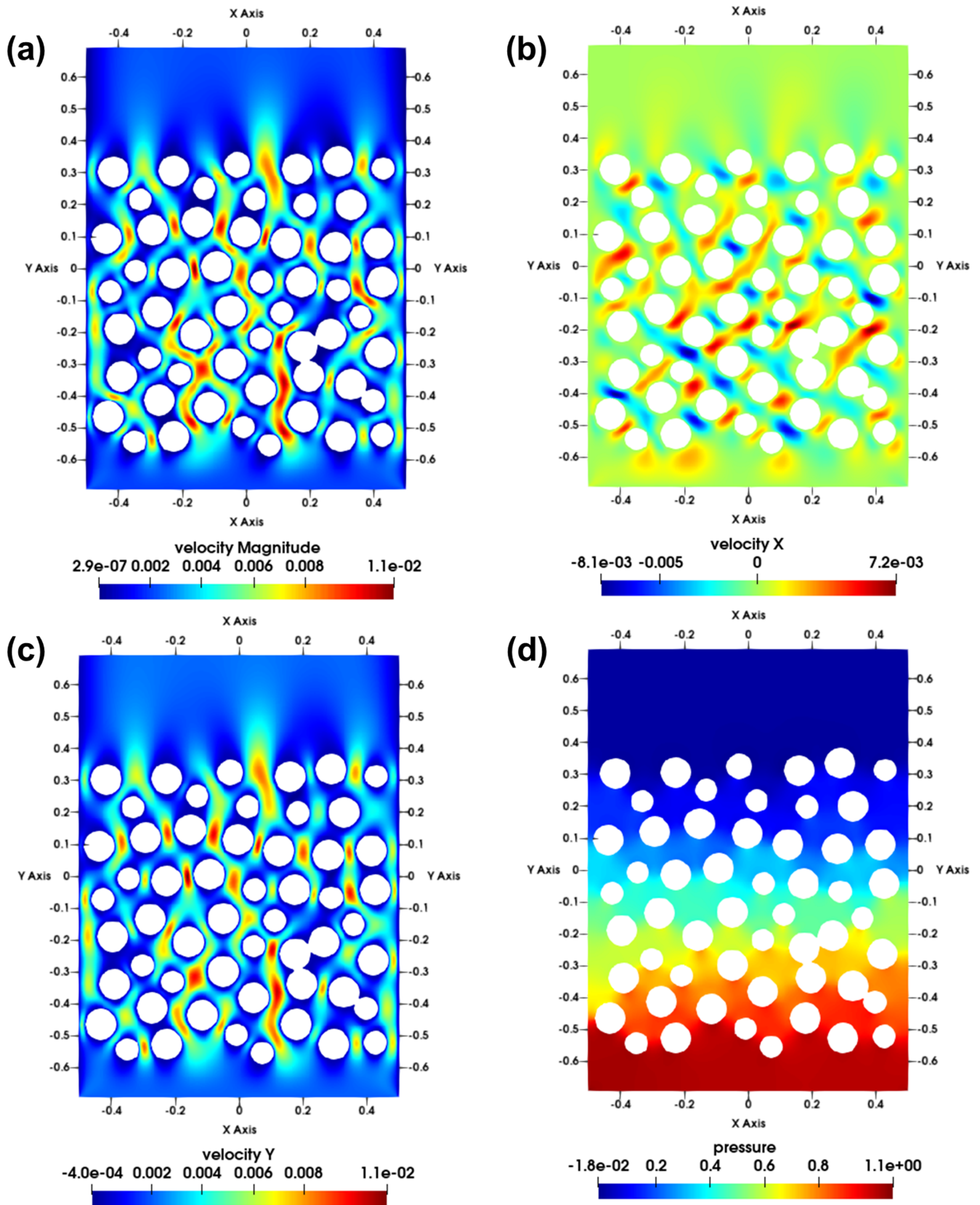


FIGURE 10 | Steady state results for 2D pipe flow with obstacles: (a) velocity magnitude, (b) x-direction velocity, (c) y-direction velocity, (d) pressure.

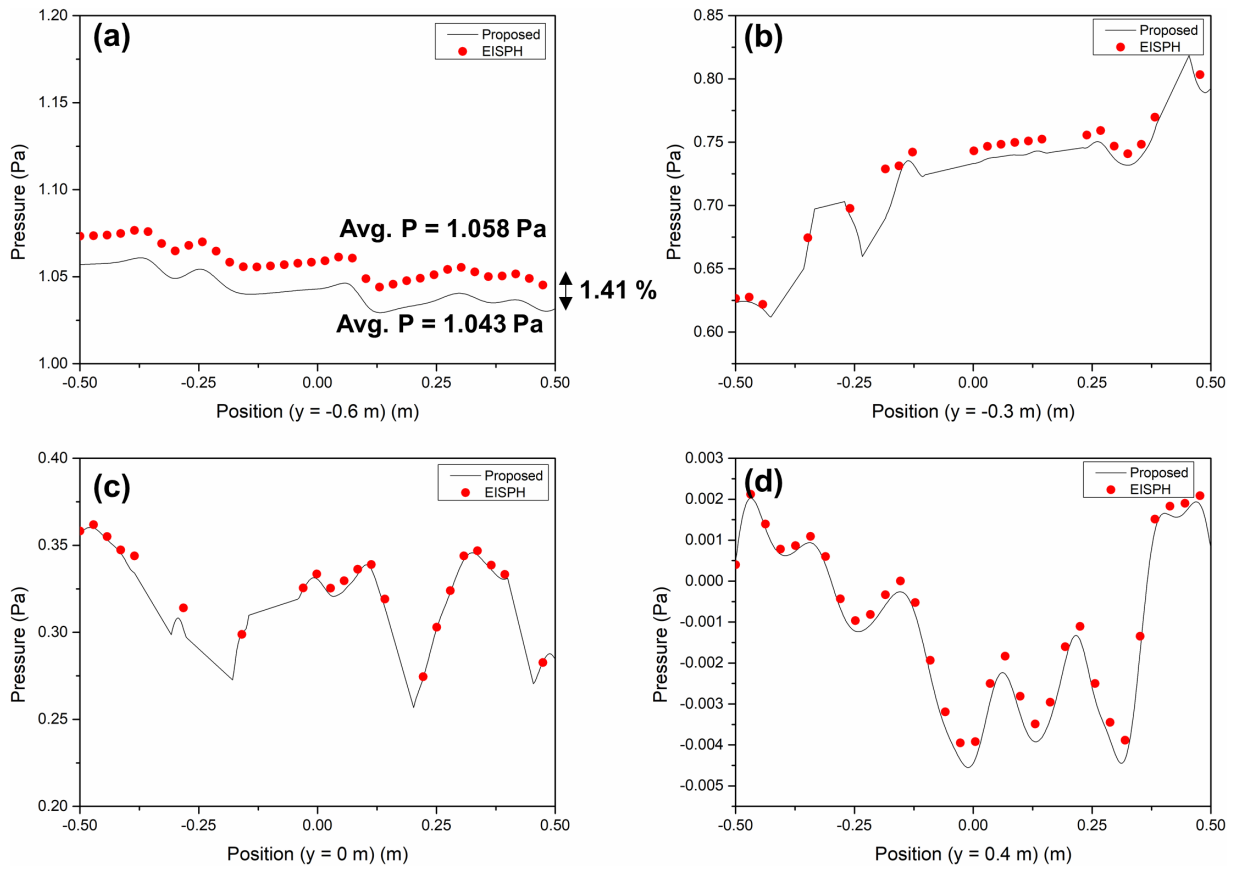


FIGURE 11 | The steady state pressure profiles at: (a) $y = -0.6$ m, (b) $y = -0.3$ m, (c) $y = 0.0$ m, (d) $y = 0.4$ m.

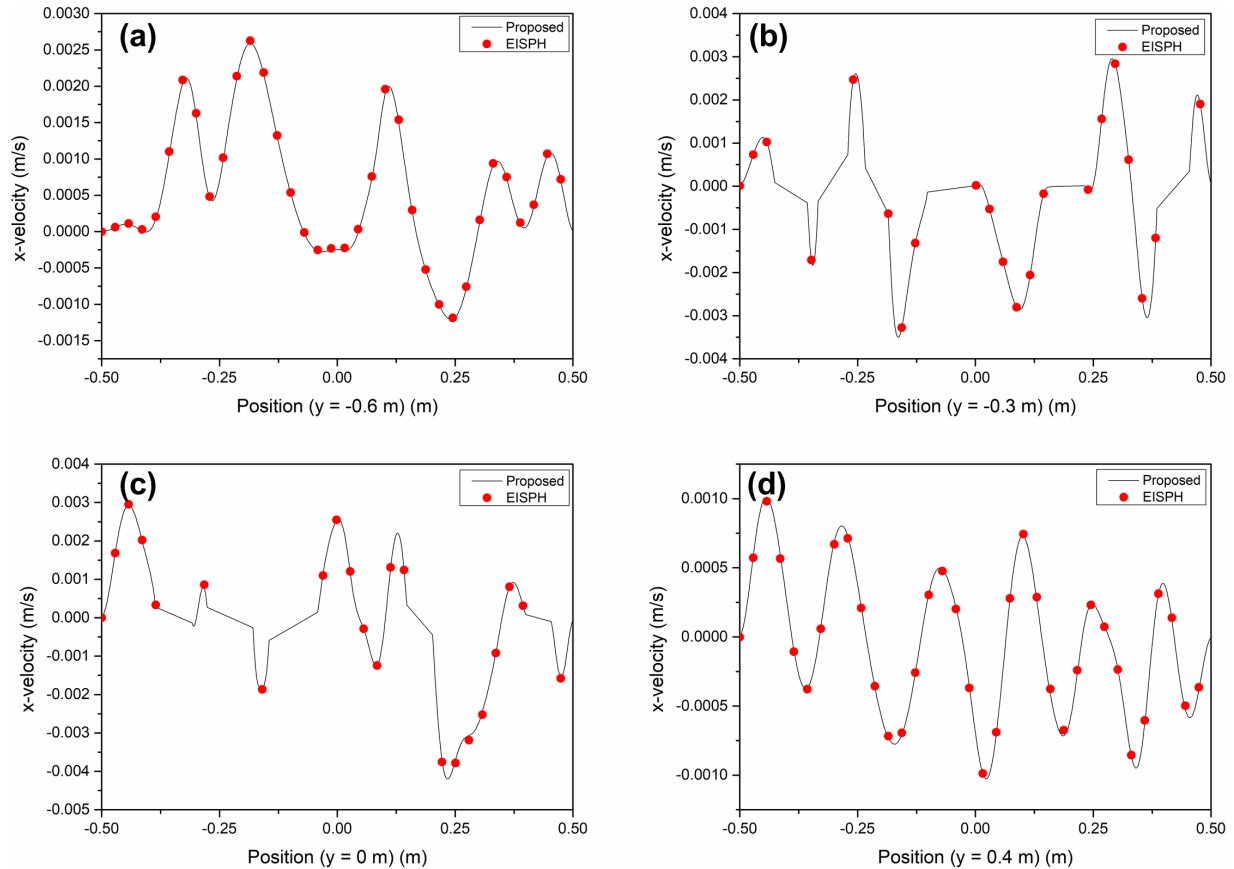


FIGURE 12 | The steady state x-direction velocity profiles at: (a) $y = -0.6$ m, (b) $y = -0.3$ m, (c) $y = 0.0$ m, (d) $y = 0.4$ m.

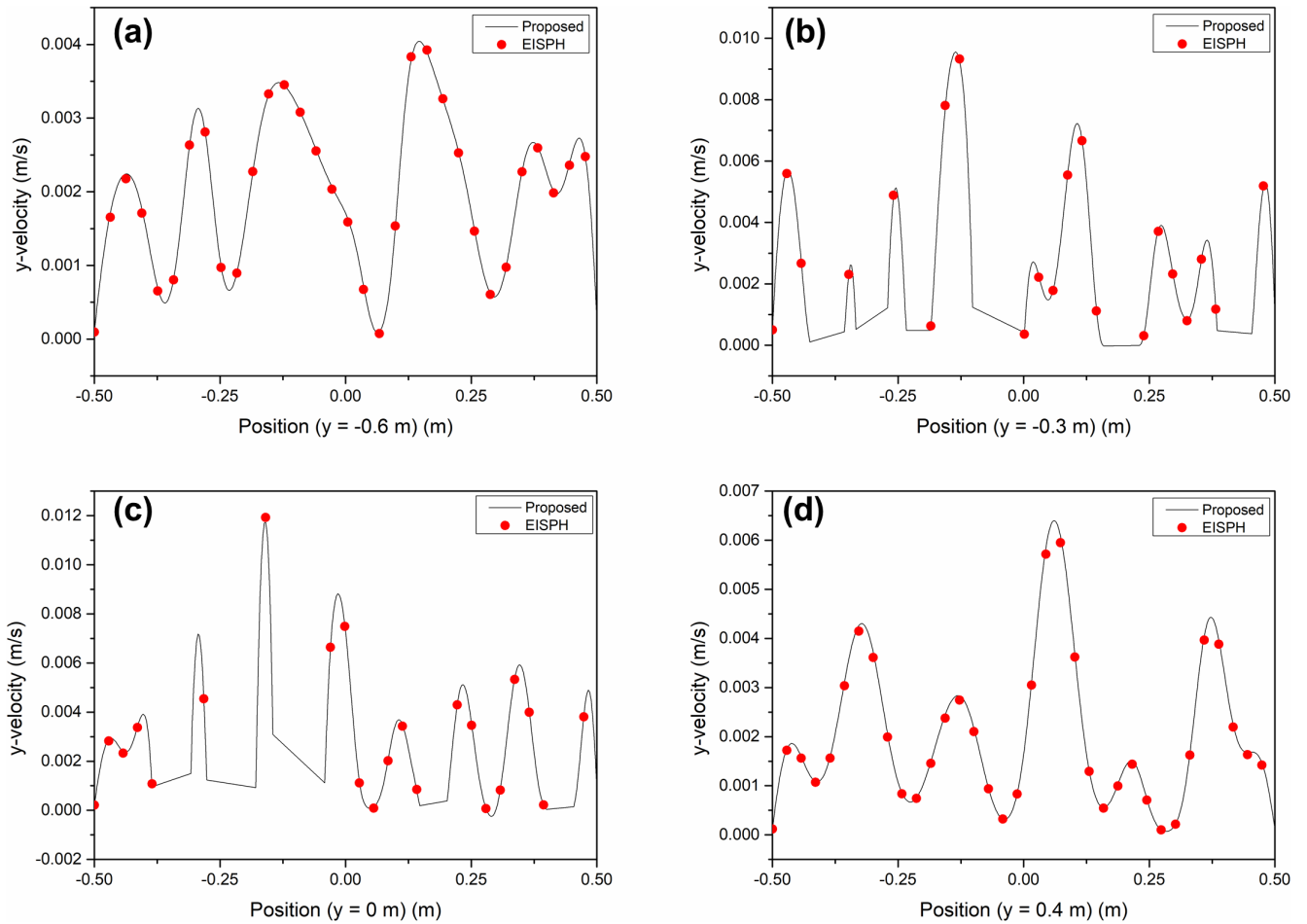


FIGURE 13 | The steady state y-direction velocity profiles at: (a) $y = -0.6$ m, (b) $y = -0.3$ m, (c) $y = 0.0$ m, (d) $y = 0.4$ m [51].

For quantitative validation, the results are compared with those obtained using EISPH, a previously developed SPH-based solver that has been successfully applied to a variety of hydrothermal problems, demonstrating reliable predictions of flow velocity and pressure drop in porous media [51]. Figures 11–13 compare the computed pressure and velocity components (x - and y -direction) with results from the EISPH method. Four characteristic lines are selected for cross-sectional validation at $y = -0.6$ m, -0.3 m, 0 m, and 0.4 m, which correspond to:

- the inlet region of the obstacle zone.
- the intersecting flow region.
- the location of peak velocity.
- the outlet region of the obstacle zone, respectively.

The pressure comparison shows a maximum deviation of 1.41% in the inlet region, while maintaining good quantitative agreement in all other locations. The x - and y -direction velocity profiles exhibit consistent agreement across all selected lines, confirming the accuracy of the proposed solver in capturing complex flow structures.

5.3 | 2D Lid Driven Flow

This section analyzes a two-dimensional lid-driven cavity flow, in which fluid motion is induced by the tangential movement of wall particles along the top boundary (Figures 14–16). The simulation is performed for a square cavity at a Reynolds number of $Re = 1000$, and both computational efficiency and quantitative accuracy are compared against results from the EISPH method [50] and ANSYS Fluent, a commercial finite-volume fluid simulation software [36].

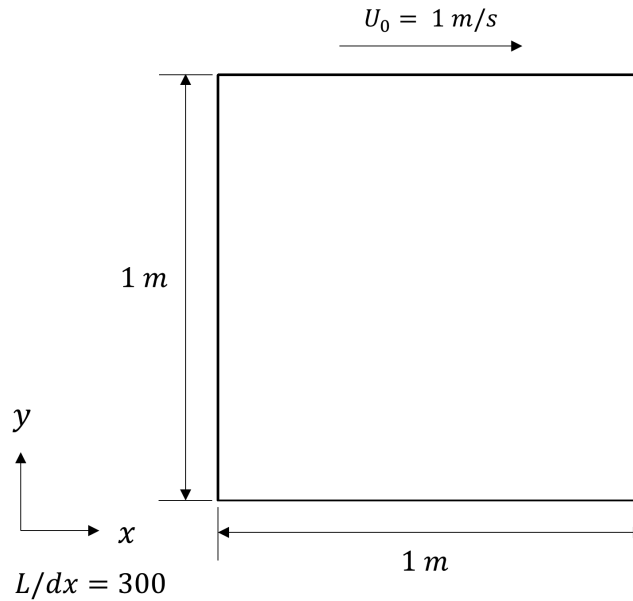


FIGURE 14 | Computational domain for the 2D lid driven flow.

The characteristic parameters are defined as $L = 1$ m, $u = 1$ m/s, $\mu = 1$ Pa · s, and $\rho = 1000$ kg/m³. The computational domain, shown in Figure 14, consists of 96,100 particles with a spatial resolution of $L/\Delta x = 300$. The proposed steady-state solver reaches convergence in 67 s, whereas EISPH requires 1163 s under a CFL number of 0.1 and time step $\Delta t = 2.04 \times 10^{-4}$ s.

Figure 15 presents the steady-state flow field results: (a) velocity magnitude, (b) x -direction velocity, (c) y -direction velocity, and (d) streamlines. The resulting streamline plot clearly captures the primary vortex located at the center of the cavity, along with secondary corner vortices forming near the lower-left and lower-right corners. These flow features are in good agreement with benchmark behavior for lid-driven cavity flows at $Re = 1000$.

Figure 16 provides a quantitative comparison of the velocity profiles: (a) normalized y -direction velocity along the horizontal centerline and (b) normalized x -direction velocity along the vertical centerline. Results are compared with reference data from Ghia et al. [52], as well as with those from EISPH and ANSYS Fluent. The velocity profiles demonstrate good agreement across all methods, further validating the accuracy of the proposed steady-state solver.

5.4 | 2D Lid Driven Flow With Heat Transfer (No Gravitational Force)

This section analyzes a two-dimensional lid-driven cavity flow with heat transfer, as illustrated in Figures 17–19. The computational performance and quantitative results of the proposed steady-state solver are compared against those obtained from the EISPH method [50] and ANSYS Fluent [36] for a square cavity with Reynolds number $Re = 1000$ and Péclet number $Pe = 100$, where $Pe = Lu\rho c_p/k$. The characteristic parameters for the problem are defined as: $L = 1$ m, $u = 1$ m/s, $\mu = 1$ Pa · s, $\rho = 1000$ kg/m³, $c_p = 0.1$ J/(kg · K), and $k = 1$ W/(m · K).

The computational domain is shown in Figure 17 and consists of 96,100 particles, corresponding to a resolution of $L/\Delta x = 300$. The steady-state solver reaches convergence in 135 s, whereas EISPH requires 1211 s under a Courant–Friedrichs–Lewy (CFL) number of 0.1 with a time step of $\Delta t = 2.04 \times 10^{-4}$ s.

Figure 18 presents the steady-state temperature distribution, including (a) the overall temperature field and (b) contour plots within the cavity. Figure 19 compares the temperature profiles along (a) the horizontal centerline and (b) the vertical centerline with results from ANSYS Fluent and EISPH. The temperature predictions of the proposed solver show good quantitative agreement with both reference methods, demonstrating its accuracy and efficiency in simulating coupled fluid-thermal flows.

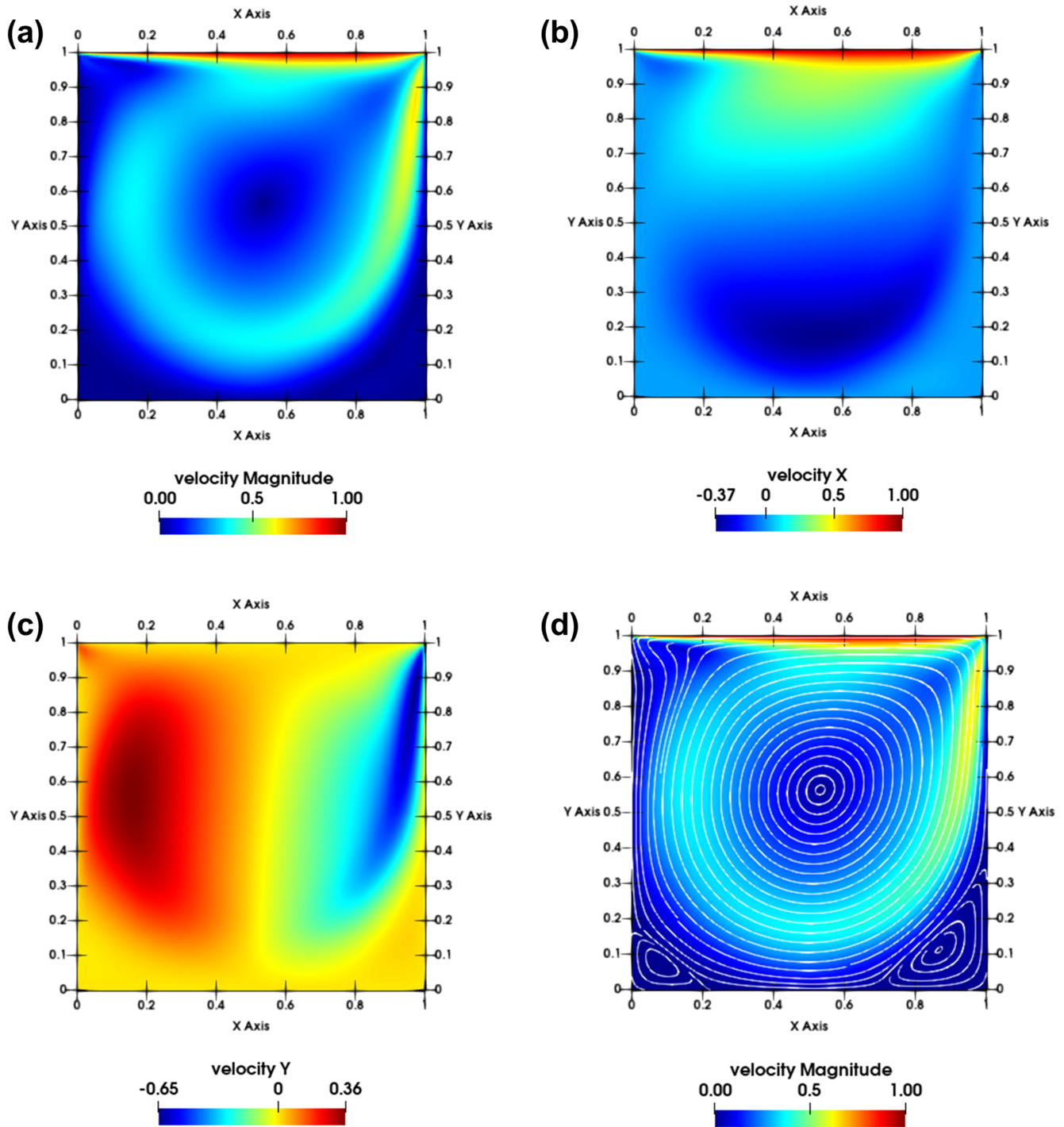


FIGURE 15 | Steady state results for 2D lid driven flow: (a) velocity magnitude, (b) x-direction velocity, (c) y-direction velocity, (d) streamlines in the velocity field.

5.5 | 2D Natural Convection Flow ($Ra = 10^4$)

This section analyzes a two-dimensional natural convection flow in a square cavity, as illustrated in Figures 20–22. The left and right walls are kept at constant temperature with $\Delta T = 10$ K, while the top and bottom walls are adiabatic boundaries. Buoyancy is modeled via the Boussinesq approximation with gravity acting in the negative y -direction. In natural convection flow, characteristic velocity is small; a velocity residual threshold of 10^{-6} is insufficient for convergence and is tightened to 10^{-7} in this simulation.

The computational performance and quantitative results of the proposed steady-state solver are compared against those obtained from the EISPH method [50] and ANSYS Fluent [36] for a square cavity with Rayleigh number $Ra = 10^4$,

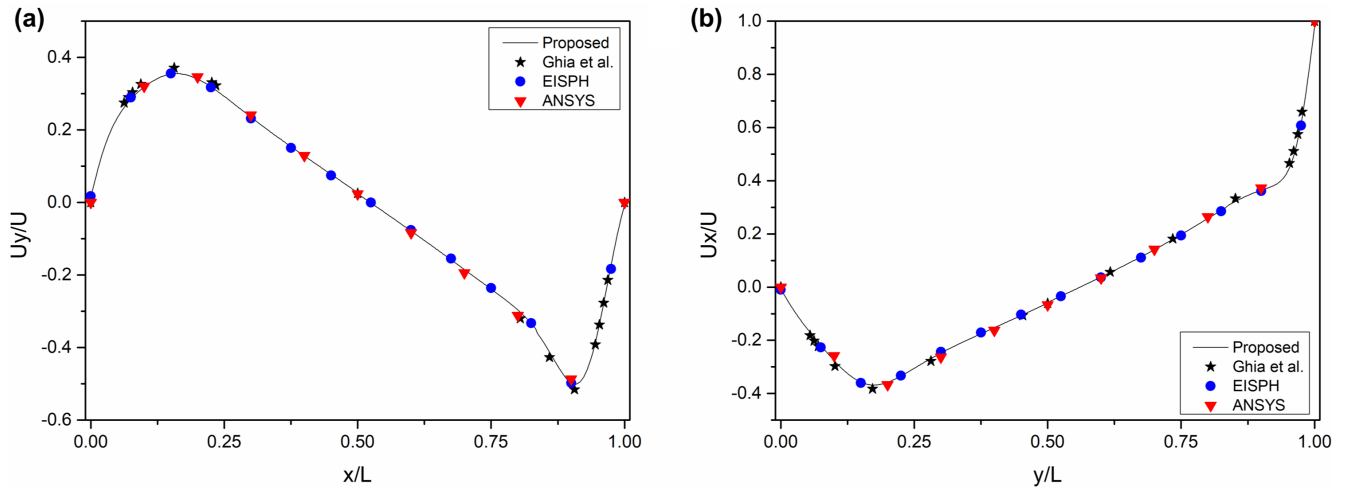


FIGURE 16 | The steady state results compared with Ghia et al. [52], EISPH [50] and ANSYS [36]: (a) the normalized y-direction velocity along the horizontal centerline, (b) the normalized x-direction velocity along the vertical centerline.

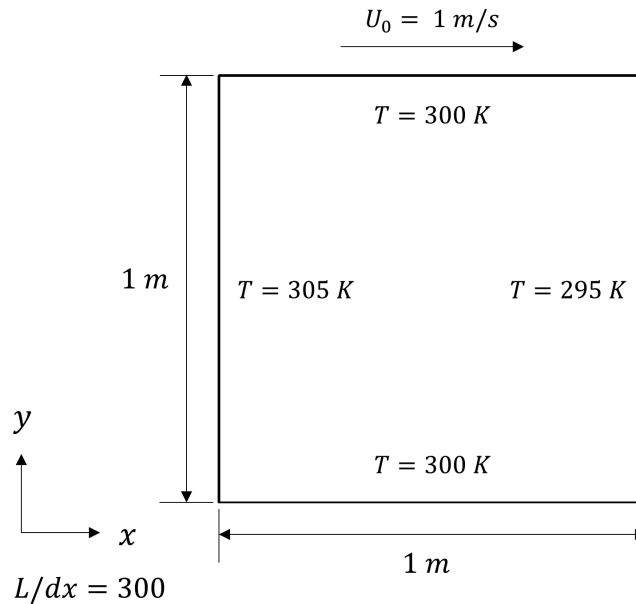


FIGURE 17 | Computational domain for the 2D lid driven flow with heat transfer.

where $Ra = g\beta\Delta TL^3/\nu\alpha$. The characteristic parameters for the problem are defined as: $L = 1\text{ m}$, $\beta = 1.44 \times 10^{-4}\text{ K}^{-1}$, $\mu = 1\text{ Pa} \cdot \text{s}$, $\rho = 1000\text{ kg/m}^3$, $c_p = 0.1\text{ J/(kg} \cdot \text{K)}$, and $k = 0.14085\text{ W/(m} \cdot \text{K)}$, these give $\nu = \mu/\rho = 10^{-3}\text{ m}^2/\text{s}$, $\alpha = k/(\rho c_p) = 1.4085 \times 10^{-3}\text{ m}^2/\text{s}$, $Pr = \nu/\alpha = 0.71$.

The computational domain is shown in Figure 20 and consists of 96,100 particles, corresponding to a resolution of $L/\Delta x = 300$. The steady-state solver reaches convergence in 94 s, whereas EISPH requires 1441 s under a Courant–Friedrichs–Lewy (CFL) number of 0.25 with a time step of $\Delta t = 5.10 \times 10^{-4}\text{ s}$.

Figure 21 presents the steady-state velocity and temperature distribution, including (a) the overall velocity magnitude field and (b) the overall temperature field within the cavity. Figure 22 compares (a) y-direction velocity profile along the horizontal centerline, (b) temperature profile along the horizontal centerline, (c) x-direction velocity profile along the vertical centerline, and (d) temperature profile along the vertical centerline with results from ANSYS Fluent and EISPH. The velocity and temperature predictions of the proposed solver show good quantitative agreement with both reference methods, demonstrating its accuracy and efficiency in simulating coupled fluid-thermal flows.

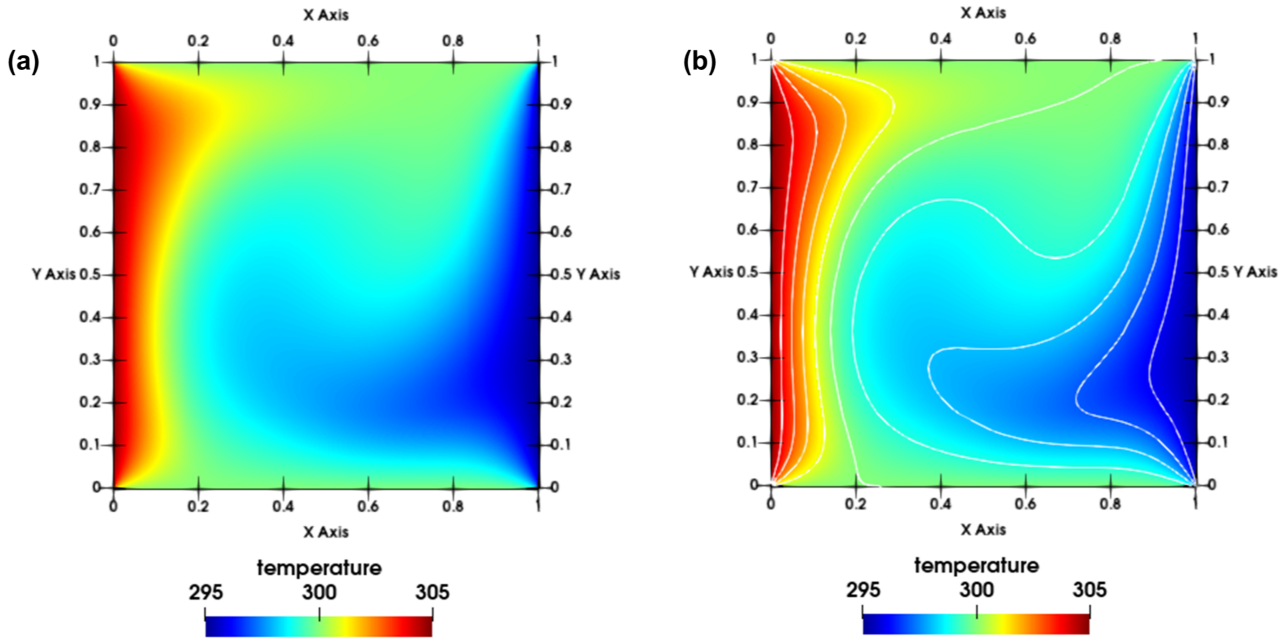


FIGURE 18 | Steady state results for 2D lid driven flow with heat transfer: (a) temperature, (b) contours in the temperature field.

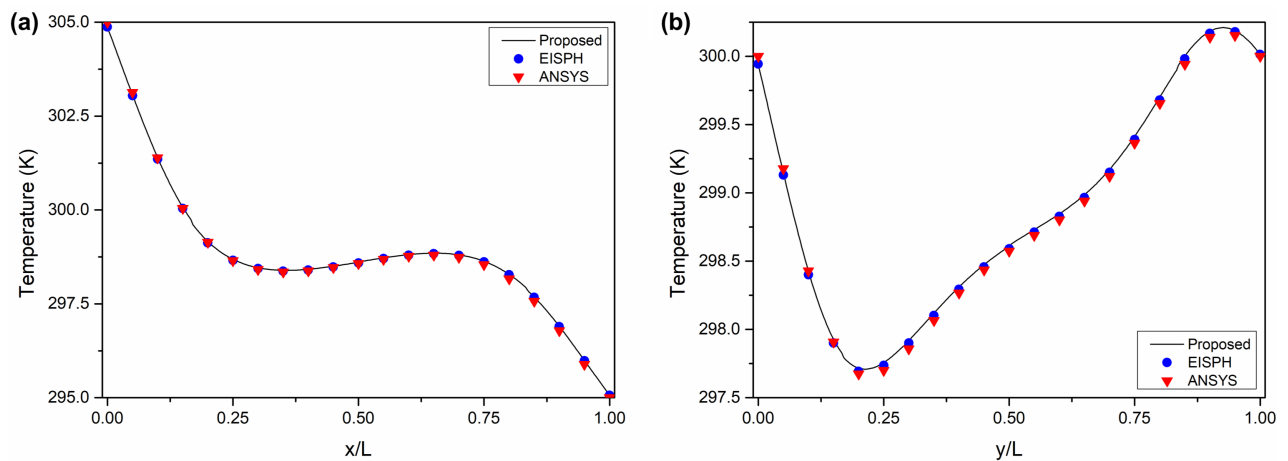


FIGURE 19 | The steady state results compared with EISPH [50] and ANSYS [36]: (a) temperature along the horizontal centerline, (b) temperature along the vertical centerline.

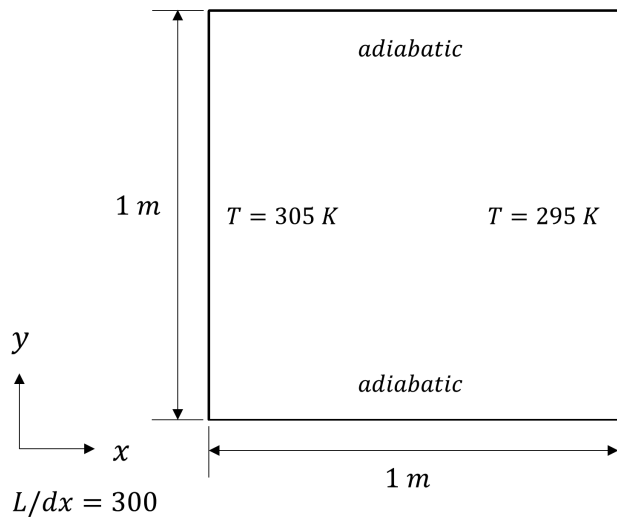


FIGURE 20 | Computational domain for the 2D natural convection flow.

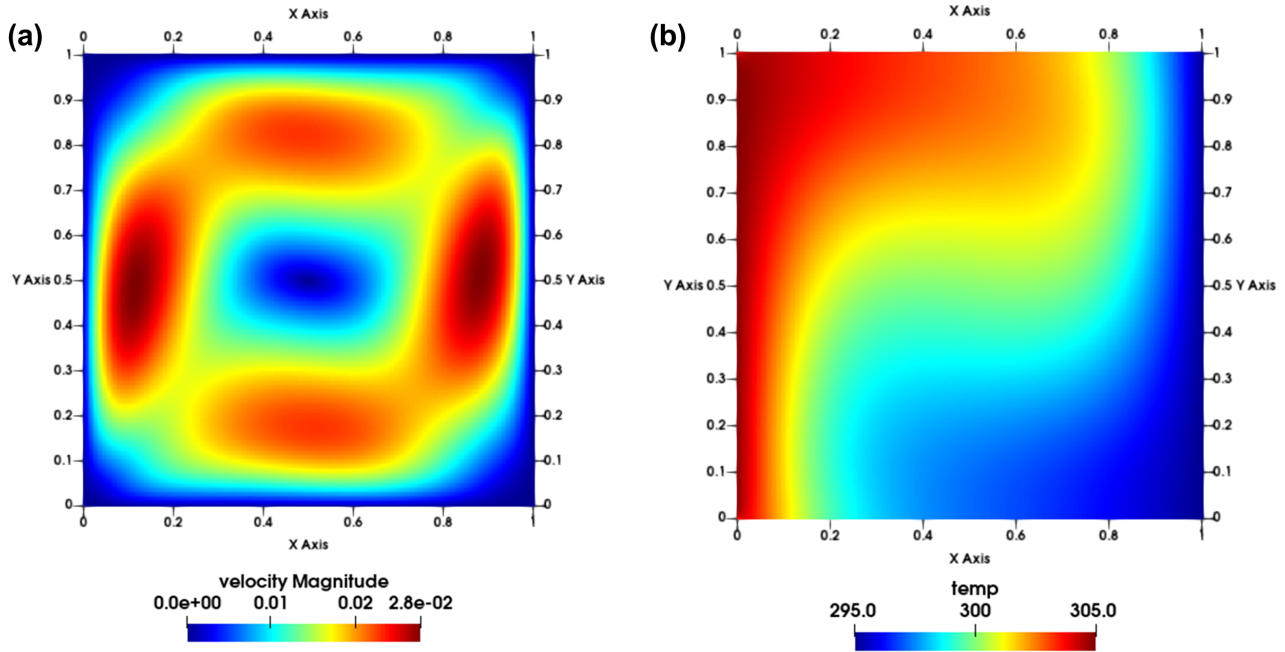


FIGURE 21 | Steady state results for the 2D natural convection flow: (a) velocity magnitude, (b) temperature.

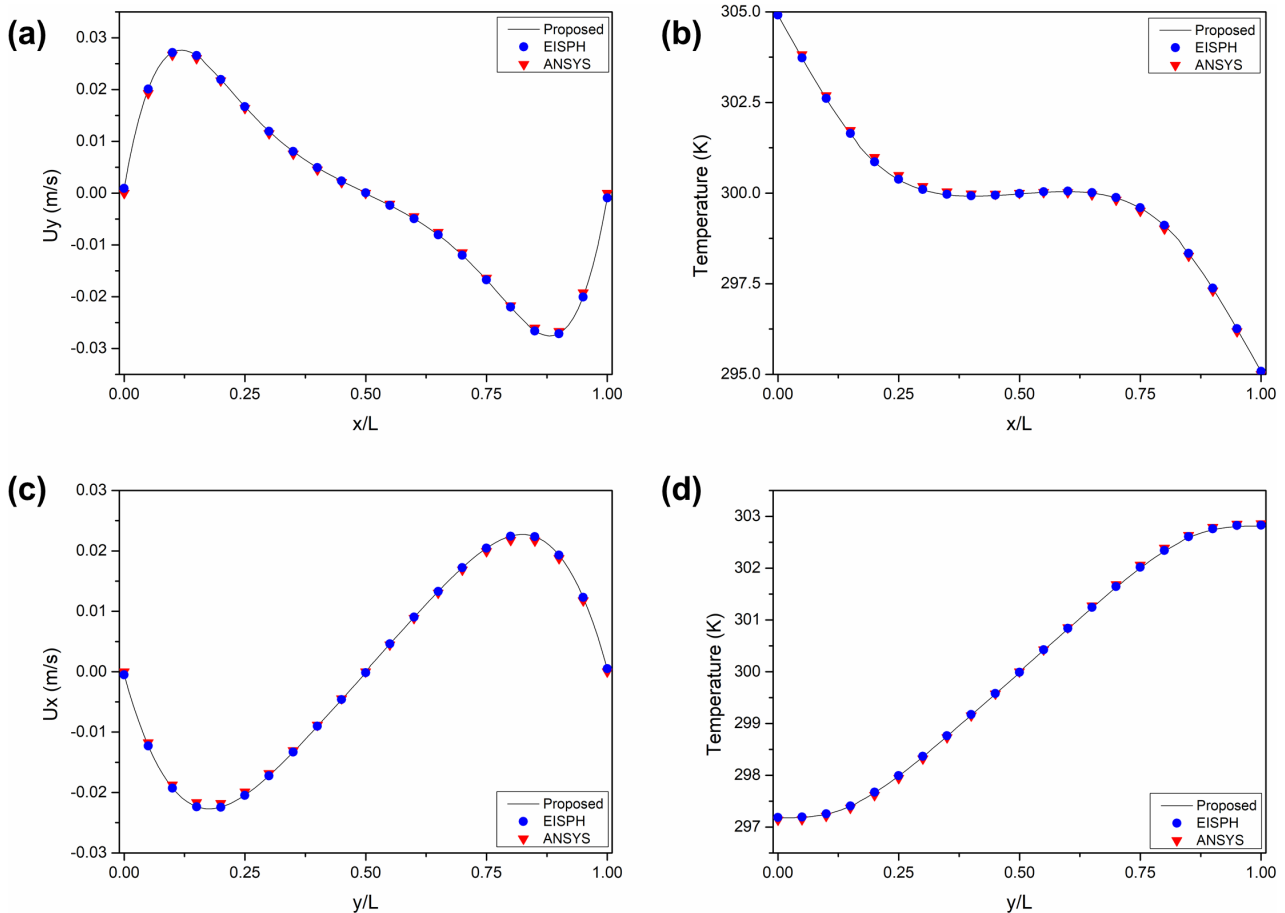


FIGURE 22 | The steady state results compared with EISPH [50] and ANSYS [36]: (a) y-direction velocity along the horizontal centerline, (b) temperature along the horizontal centerline, (c) x-direction velocity along the vertical centerline, (d) temperature along the vertical centerline.

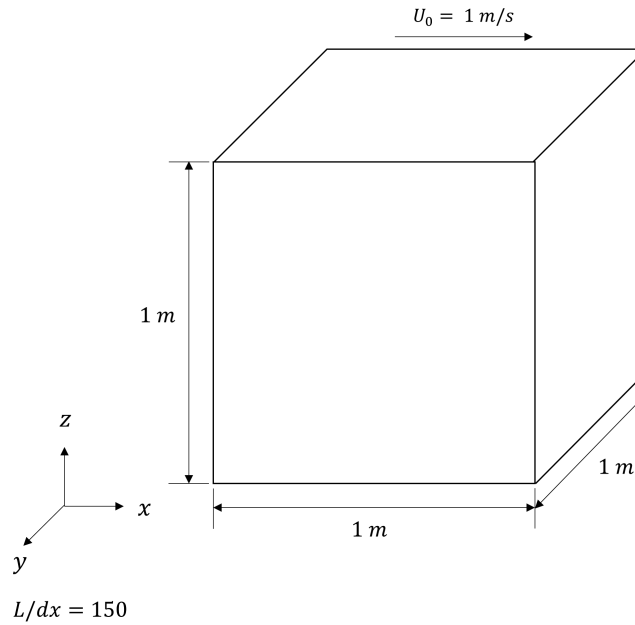


FIGURE 23 | Computational domain for the 3D lid driven flow.

5.6 | 3D Lid Driven Flow

This section analyzes a three-dimensional lid-driven cavity flow, in which fluid motion is induced by the tangential movement of wall particles along the top boundary (Figures 23–25). The simulation is performed for a square cavity at a Reynolds number of $Re = 400$, and quantitative accuracy is compared against results from the ANSYS Fluent [36] and Ku et al. [53].

The characteristic parameters are defined as $L = 1$ m, $u = 0.4$ m/s, $\mu = 1$ Pa · s, and $\rho = 1000$ kg/m³. The computational domain, shown in Figure 23, consists of 3,944,312 particles with a spatial resolution of $L/\Delta x = 150$. The proposed steady-state solver reaches convergence in 2187 s.

Figure 24 presents the steady-state flow field results: (a) volume rendering of velocity magnitude, (b) velocity magnitude on the plane normal to y -axis, (c) velocity magnitude on the plane normal to x -axis, and (d) velocity magnitude on the plane normal to z -axis. These flow features are in good agreement with benchmark behavior for lid-driven cavity flows at $Re = 400$.

Figure 25 provides a quantitative comparison of the velocity profiles: (a) normalized z -direction velocity along the x -centerline ($y/L = 0.5$, $z/L = 0.5$) and (b) normalized x -direction velocity along the z -centerline ($x/L = 0.5$, $y/L = 0.5$). Results are compared with reference data from Ghia et al. [52] (2D) and Ku et al. [53] (3D). In the 3D lid-driven cavity flow, spanwise motion and corner vortices redistribute momentum. So the velocity extrema are generally attenuated and their locations shift slightly toward the lid. The results are consistent with the 3D results of Ku et al., rather than the 2D reference of Ghia et al.

6 | Conclusions

A steady state Eulerian SPH solver based on the SIMPLE algorithm has been developed. The steady solver is formulated implicitly, allowing the computation of velocity, pressure, and temperature fields in matrix form. The Bi-CGSTAB iterative method is employed to efficiently solve the resulting matrix equations.

The governing equations are discretized and expressed in a form suitable for SPH. The Wendland 2nd kernel function is used, and the discretization is performed for both the Navier–Stokes equations and the heat conduction equation.

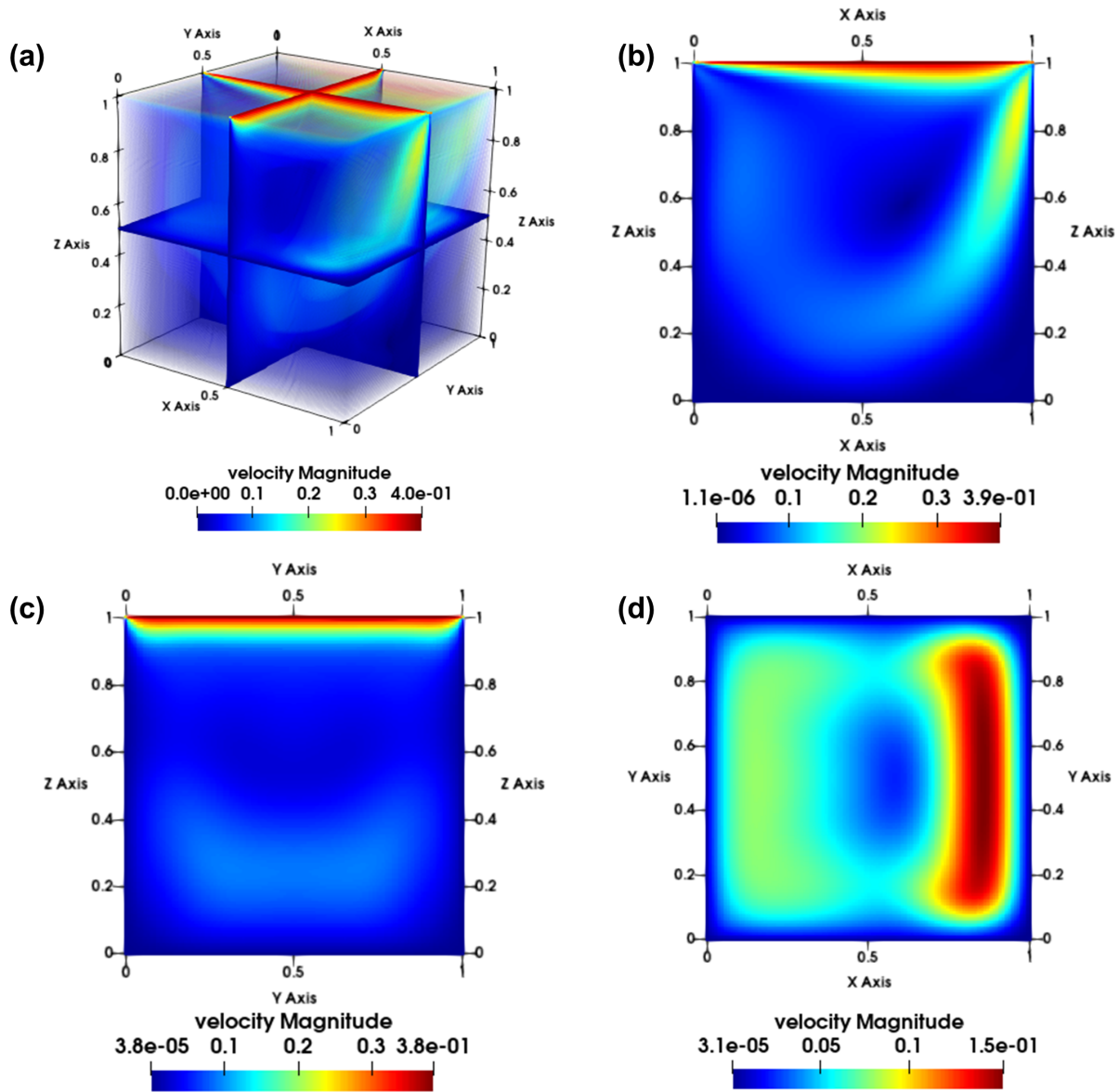


FIGURE 24 | Steady state results for the 3D lid driven flow: (a) volume rendering of velocity magnitude, (b) velocity magnitude on the plane normal to the y-axis, (c) velocity magnitude on the plane normal to the x-axis, (d) velocity magnitude on the plane normal to the z-axis.

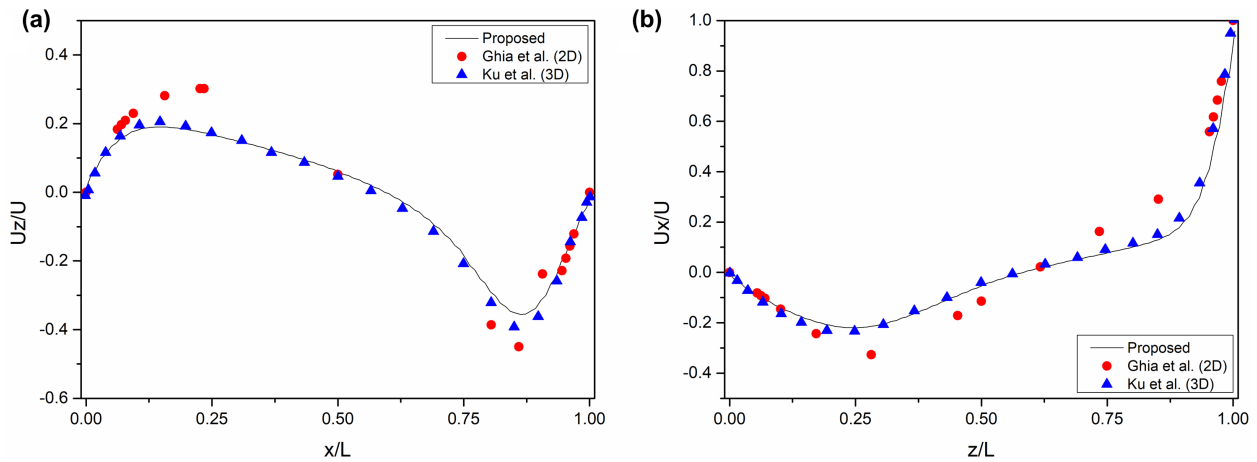


FIGURE 25 | The steady state results compared with Ghia et al. [52] (2D) and Ku et al. [53] (3D): (a) the normalized z-direction velocity along the x-centerline ($y/L = 0.5, z/L = 0.5$), (b) the normalized x-direction velocity along the z-centerline ($x/L = 0.5, y/L = 0.5$).

The steady solver updates velocity, pressure, and temperature in a unified outer iteration, implementing a two-way coupling scheme. During each step, a PPE is solved, the velocity is corrected to enforce incompressibility, and the energy equation is updated using the current velocity field; the resulting temperature then updates the Boussinesq buoyancy source for the next step momentum matrix. Convergence is checked with joint tolerances on velocity and temperature residuals, and iteration stops when both criteria are satisfied.

To enhance convergence, an under-relaxation is applied, and an appropriately defined relaxation factor is used to mitigate oscillations in physical quantities depending on the problem. Additionally, to improve computational performance, simulations are conducted using a GPU-based parallelized code.

To validate the developed steady solver, 6 benchmark tests are conducted. The first benchmark test, 2D pipe is performed to verify the proper functionality of pressure and velocity calculation loop. For flow at $Re = 10$, a quantitative comparison of the velocity profile and pressure with analytical solution is conducted, confirming accurate results. Additionally, it is observed that accuracy improves with increasing resolution.

The second benchmark test, 2D pipe flow with obstacles, is conducted to verify the proper implementation of boundary conditions in complex geometry. The velocity and pressure fields, influenced by the complex geometry, are quantitatively compared with EISPH results, confirming consistency between the solutions.

The third benchmark test, 2D lid driven flow, is conducted to verify whether the flow develops correctly when a nonzero velocity is imposed on the wall, starting from an initial condition where all fluid particles do not move. For flow at $Re = 1000$, a quantitative comparison of velocity is performed with EISPH, ANSYS and FDM results, confirming consistency among the solutions.

The fourth benchmark test, 2D lid driven flow with heat transfer (no gravitational force), is conducted to verify the proper functionality of the temperature calculation loop. For a flow at $Re = 1000$, $Pe = 100$, a quantitative comparison of temperature is performed with EISPH and ANSYS results, confirming consistency among the solutions.

The fifth benchmark test, 2D natural convection flow, is conducted to verify the proper functionality of two-way coupling scheme. For a flow at $Ra = 10^4$, a quantitative comparison of velocity and temperature is performed with EISPH and ANSYS results, confirming consistency among the solutions.

The sixth benchmark test, 3D lid driven flow, is conducted to validate the 3D implementation of the steady solver. For a flow at $Re = 400$, a quantitative comparison of velocity is performed with FDM results, confirming consistency among the solutions.

The computational time for the benchmark tests is measured, and the developed steady solver is found to be 8.97 to 17.36 times faster than obtaining the steady state flow field compared to the transient based EISPH. Notably, the steady solver demonstrated superior efficiency in flow problems involving complex geometries, highlighting its advantage in such cases.

The Eulerian SPH steady solver developed in this study is capable of rapidly obtaining steady state flow fields and can also be utilized as an initial input for transient simulations. In particular, it is expected to be a powerful tool for analyzing real-world phenomena by initiating from a steady state condition.

Acknowledgments

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (no. 2021M2D2A1A03046881, RS-2024-00436693).

Funding

This work was supported by National Research Foundation of Korea (2021M2D2A1A03046881, RS-2024-00436693).

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The data that support the findings of this study are available from the corresponding author upon reasonable request.

References

1. L. B. Lucy, "A Numerical Approach to the Testing of the Fission Hypothesis," *Astronomical Journal* 82 (1977): 1013–1024.
2. R. A. Gingold and J. J. Monaghan, "Smoothed Particle Hydrodynamics: Theory and Application to Non-Spherical Stars," *Monthly Notices of the Royal Astronomical Society* 181, no. 3 (1977): 375–389.
3. J. J. Monaghan, "Smoothed Particle Hydrodynamics," *Annual Review of Astronomy and Astrophysics* 30 (1992): 543–574.
4. H.-G. Lyu, P.-N. Sun, X.-T. Huang, et al., "A Review of SPH Techniques for Hydrodynamic Simulations of Ocean Energy Devices," *Energies* 15, no. 2 (2022): 502.
5. J. J. Monaghan, "Smoothed Particle Hydrodynamics and Its Diverse Applications," *Annual Review of Fluid Mechanics* 44, no. 1 (2012): 323–346.
6. D. Violeau and B. D. Rogers, "Smoothed Particle Hydrodynamics (SPH) for Free-Surface Flows: Past, Present and Future," *Journal of Hydraulic Research* 54, no. 1 (2016): 1–26.
7. T. Ye, D. Pan, C. Huang, and M. Liu, "Smoothed Particle Hydrodynamics (SPH) for Complex Fluid Flows: Recent Developments in Methodology and Applications," *Physics of Fluids* 31, no. 1 (2019): 011301.
8. J. J. Monaghan, "Simulating Free Surface Flows With SPH," *Journal of Computational Physics* 110, no. 2 (1994): 399–406.
9. Z. Chen, Z. Zong, M. Liu, L. Zou, H. Li, and C. Shu, "An SPH Model for Multiphase Flows With Complex Interfaces and Large Density Differences," *Journal of Computational Physics* 283 (2015): 169–188.
10. D. Le Touzé and A. Colagrossi, "Smoothed Particle Hydrodynamics for Free-Surface and Multiphase Flows: A Review," *Reports on Progress in Physics* (2025): 037001.
11. M. Afrasiabi, C. Lüthi, M. Bambach, and K. Wegener, "Multi-Resolution SPH Simulation of a Laser Powder Bed Fusion Additive Manufacturing Process," *Applied Sciences* 11, no. 7 (2021): 2962.
12. F. He, Y. Chen, Y. Zhang, H. Jiang, and C. Huang, "A Multi-Physics SPH Method for Simulating Heat Transfer, Mass Transfer and Phase Transition of Hydrate," *International Communications in Heat and Mass Transfer* 165 (2025): 109012.
13. M. Liu and Z. Zhang, "Smoothed Particle Hydrodynamics (SPH) for Modeling Fluid-Structure Interactions," *Science China Physics, Mechanics & Astronomy* 62 (2019): 1–38.
14. Y. Ma, X. Zhou, F. Zhang, C. Weißenfels, and M. Liu, "A Novel Smoothed Particle Hydrodynamics Method for Multi-Physics Simulation of Laser Powder Bed Fusion," *Computational Mechanics* 74, no. 5 (2024): 1009–1036.
15. E. Majtan, L. S. Cunningham, and B. D. Rogers, "Investigation of Flooded Bridge Piers Using a Coupled SPH-DEM Approach," 2025, International Brick and Block Masonry Conference.
16. H. S. Yoo and E. S. Kim, "Numerical Investigation of Spreading Phenomena of Molten Corium Using EISPH Method," *Nuclear Engineering and Technology* 57, no. 4 (2025): 103301.
17. J. J. Monaghan and R. A. Gingold, "Shock Simulation by the Particle Method SPH," *Journal of Computational Physics* 52, no. 2 (1983): 374–389.
18. V. Springel, "Smoothed Particle Hydrodynamics in Astrophysics," *Annual Review of Astronomy and Astrophysics* 48, no. 1 (2010): 391–430.
19. M. Antuono, A. Colagrossi, and S. Marrone, "Numerical Diffusive Terms in Weakly-Compressible SPH Schemes," *Computer Physics Communications* 183, no. 12 (2012): 2570–2580.
20. D. Molteni and A. Colagrossi, "A Simple Procedure to Improve the Pressure Evaluation in Hydrodynamic Context Using the SPH," *Computer Physics Communications* 180, no. 6 (2009): 861–872.
21. J. Michel, M. Antuono, G. Oger, and S. Marrone, "Energy Balance in Quasi-Lagrangian Riemann-Based SPH Schemes," *Computer Methods in Applied Mechanics and Engineering* 410 (2023): 116015.
22. C. Zhang, X. Hu, and N. A. Adams, "A Weakly Compressible SPH Method Based on a Low-Dissipation Riemann Solver," *Journal of Computational Physics* 335 (2017): 605–620.
23. S. J. Cummins and M. Rudman, "An SPH Projection Method," *Journal of Computational Physics* 152, no. 2 (1999): 584–607.
24. R. Xu, P. Stansby, and D. Laurence, "Accuracy and Stability in Incompressible SPH (ISPH) Based on the Projection Method and a New Approach," *Journal of Computational Physics* 228, no. 18 (2009): 6703–6725.
25. X. Hu and N. A. Adams, "An Incompressible Multi-Phase SPH Method," *Journal of Computational Physics* 227, no. 1 (2007): 264–278.

26. S. V. Patankar and D. B. Spalding, "A Calculation Procedure for Heat, Mass and Momentum Transfer in Three-Dimensional Parabolic Flows," in *Numerical Prediction of Flow, Heat Transfer, Turbulence and Combustion* (Elsevier, 1983), 54–73.
27. S. Patankar, *Numerical Heat Transfer and Fluid Flow* (CRC Press, 2018).
28. J. P. Van Doormaal and G. D. Raithby, "Enhancements of the SIMPLE Method for Predicting Incompressible Fluid Flows," *Numerical Heat Transfer* 7, no. 2 (1984): 147–163.
29. D. Spalding, "Mathematical Modelling of Fluid-Mechanics, Heat-Transfer and Chemical-Reaction Processes," 1980, A Lecture Course, Imperial College CFDU Report.
30. R. I. Issa, "Solution of the Implicitly Discretised Fluid Flow Equations by Operator-Splitting," *Journal of Computational Physics* 62, no. 1 (1986): 40–65.
31. H. Wang, H. Wang, F. Gao, P. Zhou, and Z. J. Zhai, "Literature Review on Pressure–Velocity Decoupling Algorithms Applied to Built-Environment CFD Simulation," *Building and Environment* 143 (2018): 671–678.
32. D. Jang, R. Jetli, and S. Acharya, "Comparison of the PISO, SIMPLER, and SIMPLEC Algorithms for the Treatment of the Pressure-Velocity Coupling in Steady Flow Problems," *Numerical Heat Transfer* 10, no. 3 (1986): 209–228.
33. Z. Qu, H. Xu, and W. Tao, "Numerical Simulation of Non-Equilibrium Conjugate Heat Transfer in Tubes Partially Filled With Metallic Foams," *Journal of Thermal Science and Technology* 7, no. 1 (2012): 151–165.
34. P. Liu, N. Zheng, F. Shan, Z. Liu, and W. Liu, "Numerical Study on Characteristics of Heat Transfer and Friction Factor in a Circular Tube With Central Slant Rods," *International Journal of Heat and Mass Transfer* 99 (2016): 268–282.
35. N. Zheng, P. Liu, F. Shan, Z. Liu, and W. Liu, "Heat Transfer Enhancement in a Novel Internally Grooved Tube by Generating Longitudinal Swirl Flows With Multi-Vortexes," *Applied Thermal Engineering* 95 (2016): 421–432.
36. Ansys Inc., *ANSYS Fluent User's Guide*, 1st ed. (ANSYS Inc., 2023).
37. A. Tayebi and B. G. Dehkordi, "Development of a PISO-SPH Method for Computing Incompressible Flows," *Proceedings of the Institution of Mechanical Engineers, Part C: Journal of Mechanical Engineering Science* 228, no. 3 (2014): 481–490.
38. S. Liu, X. He, W. Wang, and E. Wu, "Adapted SIMPLE Algorithm for Incompressible SPH Fluids With a Broad Range Viscosity," *IEEE Transactions on Visualization and Computer Graphics* 28, no. 9 (2021): 3168–3179.
39. J. H. Ferziger, M. Perić, and R. L. Street, *Computational Methods for Fluid Dynamics* (Springer, 2019).
40. T. O. Foundation, "OpenFOAM User Guide," 2023, <https://www.openfoam.com/documentation/>.
41. H. Wendland, "Piecewise Polynomial, Positive Definite and Compactly Supported Radial Functions of Minimal Degree," *Advances in Computational Mathematics* 4 (1995): 389–396.
42. M. Liu and G. Liu, "Smoothed Particle Hydrodynamics (SPH): An Overview and Recent Developments," *Archives of Computational Methods in Engineering* 17 (2010): 25–76.
43. J. P. Morris, P. J. Fox, and Y. Zhu, "Modeling Low Reynolds Number Incompressible Flows Using SPH," *Journal of Computational Physics* 136, no. 1 (1997): 214–226.
44. G. Fourtakas, P. Stansby, B. D. Rogers, and S. Lind, "An Eulerian–Lagrangian Incompressible SPH Formulation (ELI-SPH) Connected With a Sharp Interface," *Computer Methods in Applied Mechanics and Engineering* 329 (2018): 532–552.
45. S. Adami, X. Y. Hu, and N. A. Adams, "A Generalized Wall Boundary Condition for Smoothed Particle Hydrodynamics," *Journal of Computational Physics* 231, no. 21 (2012): 7057–7075.
46. Y. Saad, *Iterative Methods for Sparse Linear Systems* (SIAM, 2003).
47. H. A. Van der Vorst, "Bi-CGSTAB: A Fast and Smoothly Converging Variant of bi-CG for the Solution of Nonsymmetric Linear Systems," *SIAM Journal on Scientific and Statistical Computing* 13, no. 2 (1992): 631–644.
48. Y. B. Jo, S.-H. Park, H. Y. Choi, H.-W. Jung, Y.-J. Kim, and E. S. Kim, "SOPHIA: Development of Lagrangian-Based CFD Code for Nuclear Thermal-Hydraulics and Safety Applications," *Annals of Nuclear Energy* 124 (2019): 132–149.
49. G.-R. Liu and M. B. Liu, *Smoothed Particle Hydrodynamics: A Meshfree Particle Method* (World Scientific, 2003).
50. H. S. Yoo, Y. B. Jo, and E. S. Kim, "Comparative Study of WCSPH, EISPH and Explicit Incompressible-Compressible SPH (EICSPH) for Multi-Phase Flow With High Density Difference," *Journal of Computational Physics* 506 (2024): 112930.
51. J.-W. Kim, S.-J. Yoon, T.-S. Yoo, and E. S. Kim, "Modelling and Analysis of Salt-Convection Effect on Oxide Reduction Process for Uranium Oxides Using Smoothed Particle Hydrodynamics," *International Journal of Heat and Mass Transfer* 206 (2023): 123965.

52. U. Ghia, K. N. Ghia, and C. Shin, "High-Re Solutions for Incompressible Flow Using the Navier-Stokes Equations and a Multigrid Method," *Journal of Computational Physics* 48, no. 3 (1982): 387–411.
53. H. C. Ku, R. S. Hirsh, and T. D. Taylor, "A Pseudospectral Method for Solution of the Three-Dimensional Incompressible Navier-Stokes Equations," *Journal of Computational Physics* 70, no. 2 (1987): 439–462.
54. A. Tafuni, J. Domínguez, R. Vacondio, and A. Crespo, "A Versatile Algorithm for the Treatment of Open Boundary Conditions in Smoothed Particle Hydrodynamics GPU Models," *Computer Methods in Applied Mechanics and Engineering* 342 (2018): 604–624.